

第3章 进程同步与通信

- 3.1 进程同步的基本概念
- 3.2 信号量机制
- 3.3 信号量临界区保护
- 3.4 死锁处理
- 3.5 管程机制
- 3.6 经典互斥与同步问题
- 3.7 进程通信

回顾

- 2.1 进程的概念

- PCB+程序

- 2.2 多进程图像

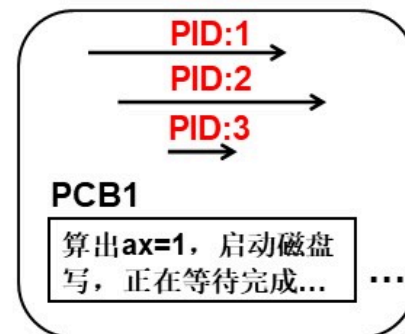
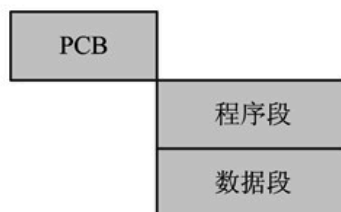
- 状态图：两态、三态、五态

- 2.3 CPU调度策略

- 三级调度：低级（进程），中级（交换），高级（作业）。
- 调度原则：时间（响应，周转，截止）
- 算法：FCFS, SJF, RR, HRRF（高响应比优先），实时。

- 2.4 线程

- 用户级线程
- 内核级线程



单选题 1分

以下对进程的描述中，错误的是

- ☐ A 进程是动态的概念
- ☐ B 进程执行需要CPU
- ☐ C 进程是有生命期的
- ☐ D 进程是指令的集合

单选题 1分

以下关于进程的描述中，正确的是

- ☐ A 进程获得CPU运行是通过调度得到的
- ☐ B 优先级是进程调度的重要依据，一旦确定就不能改变
- ☐ C 在单CPU系统中，任何时刻都有一个进程处于运行状态
- ☐ D 进程申请CPU得不到满足时，其状态变为阻塞

单选题 1分

当一个进程处于正等着_____状态时，称为阻塞状态。

- ☐ A 等待协作进程的消息
- ☐ B 进程调度
- ☐ C 分给它一个时间片
- ☐ D 进入内存

单选题 1分

在单CPU 的多进程系统中，进程切换时什么时候占用CPU 以及占用多长时间取决于_____

- ☐ A 进程相应程序段的长度
- ☐ B 进程总共需要运行时间的多少
- ☐ C 进程自身和进程调度策略
- ☐ D 进程完成什么功能

单选题 1分

下面关于线程的叙述中，正确的是_____

- ☐ A 线程是比进程更小的能够独立运行的基本单位
- ☐ B 引入线程可以提高程序并发执行的程度，可以进一步提高系统的效率。
- ☐ C 线程的引入增加了程序执行的时空开销。
- ☐ D 一个进程一定包含多个线程

第3章 进程同步与通信

- 3.1 进程同步的基本概念
- 3.2 信号量机制
- 3.3 信号量临界区保护
- 3.4 死锁处理
- 3.5 管程机制
- 3.6 经典互斥与同步问题
- 3.7 进程通信

3.1 进程同步的基本概念



实例1:

等待

```
司机
while (true){
  启动车辆;
  正常运行;
  到站停车;
```

等待

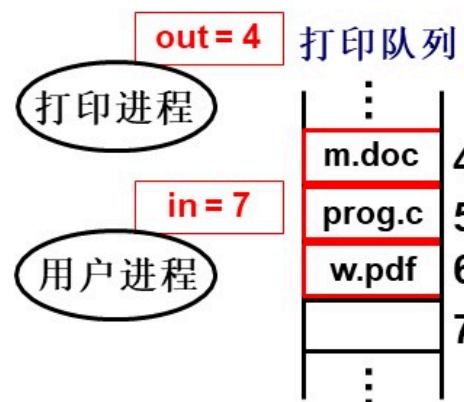
```
售票员
while (true){
  关门;
  售票;
  开门;
```

若干进程为完成一个共同的任务而相互合作，这就需要相互合作的进程在某些协调点处（即需要同步的地方）插入对信号量的P操作或V操作，以便协调它们的工作（实现进程间的同步）。

实例2: 文档打印

问题：如果进程之间完全不知道对方的存在，则可能会产生怎样的错误？

生活常识是怎么办？



3.1 进程同步的基本概念

为了协调进程之间的制约关系，达到资源共享和进程合作，需要实现进程的互斥与同步。

协调进程之间的关系是通过进程之间的通信机制来实现的，低级通信主要是通过控制进程的执行速度来保证进程之间的互斥与同步，高级通信要在进程之间传送大量的数据。

• 并发进程的关系

- (1) 间接制约关系。一组（两个或多个）进程共享一种资源，且该资源一次仅允许一个进程使用；当一个进程正在访问或使用该资源时，就制约其他进程对该资源的访问或使用，否则就可能造成执行结果的错误。
- (2) 直接制约关系。是由任务协作引起的，几个进程相互协作完成一项任务，这些进程因任务性质的要求必须按事先规定好的顺序依次执行，才能使任务得到正确的处理，否则就可能造成错误的结果。

3.1 进程同步的基本概念

- 进程的互斥与同步

- (1) 进程同步: 若干进程为完成一个共同的任务而相互合作等待对方消息的协调关系称为进程同步。
- (2) 进程互斥: 进程之间争夺互斥资源而引起的。
- (3) 饥饿: 一个就绪进程所申请的资源总是被优先于自己的其他进程所占有, 始终处于不能被调度执行的状态。
- (4) 死锁。一个进程集合中已经占有部分资源的两个或两个以上进程, 还需要获得已被其他进程占有的资源才能够继续执行; 有可能出现某些进程相互之间都在等待对方的资源且都无法运行的状态。

同步机制应遵循的准则

- (1) 空闲让进。无进程处于临界区时意味着临界资源处于空闲状态，这时若有进程要求进入临界区应立即允许进入。
- (2) 忙则等待。当已有进程进入其临界区时则意味着某临界资源正在使用，所有其他欲访问该临界资源的进程试图进入各自临界区时必须等待，以保证各进程互斥地进入访问相同临界资源的临界区。
- (3) 有限等待。若干进程要求进入访问同一临界资源的临界区时，应在有限时间内使一进程进入临界区，即不应出现各进程相互等待而都无法进入临界区的情况。
- (4) 让权等待。当进程不能进入其临界区时应立即释放所占有的CPU，以免陷入“忙等”（进程在占有CPU的同时又一直等待），保证其他可执行的进程获得CPU运行。

同步与互斥的比较

- 相似处：进程互斥实际上是进程同步的一种特殊情况，即逐次使用互斥资源，这也是对进程使用资源次序的一种协调。因此，可以将进程互斥和进程同步统称为进程同步。
- 区别是：同步是控制进程之间的协作关系，让它们能够有序地执行；而互斥是控制进程之间的竞争关系，让它们能够有序地访问共享资源。
- 在操作系统中，同步和互斥常常同时使用，以确保进程能够协调工作并避免资源冲突。

填空题 4分

同步与互斥的比较

下面活动中，每个活动分属于同步与互斥关系的哪一种，并说明其理由。

- (1) 若干同学去图书馆借书； [填空1]
- (2) 两队举行篮球比赛； [填空2]
- (3) 流水线生产的各道工序； [填空3]
- (4) 商品生产和社会消费。 [填空4]

从纸上到实际：生产者-消费者实例

共享数据

```
#define BUFFER_SIZE 10
typedef struct { ... }
item; item
buffer[BUFFER_SIZE];
int in = out = counter = 0;
```

注意：这些都是用
户态程序！

生产者进程

```
while (true) {
    while(counter == BUFFER_SIZE)
        ;
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

消费者进程

```
while (true) {
    while(counter == 0)
        ;
    item = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
}
```

找到哪些地方要停，什么时候再走？

■ 需要让“进程走走停停”来保证多进程合作的合理有序

生产者进程

消费者进程

这就是进程同步

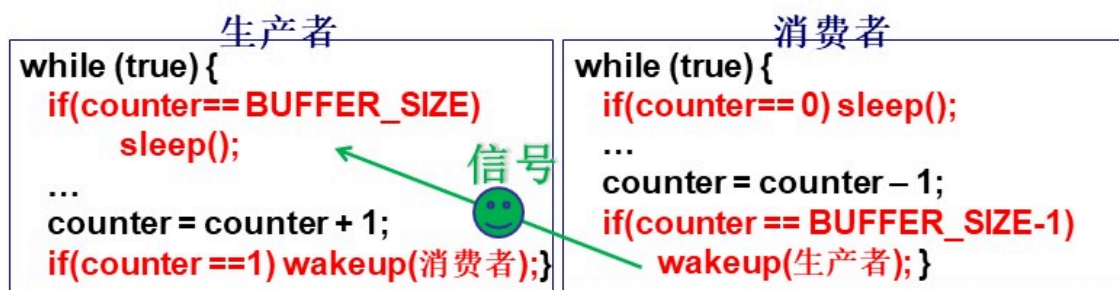
```
while (true) {  
    while(counter == BUFFER_SIZE)  
        缓存区满，生产者要停；  
    buffer[in] = item;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

发信号让消费者再走...

```
while (true) {  
    while(counter == 0)  
        缓存区空，消费者要停；  
    item = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
}
```

发信号让生产者再走...

只发信号还不能解决全部问题



- (1) 缓冲区满以后生产者 P_1 生产一个item放入，会sleep
- (2) 又一个生产者 P_2 生产一个item放入，会sleep
- (3) 消费者 C 执行1次循环， $counter==BUFFER_SIZE-1$ ，发信号给 P_1 ， P_1 wakeup
- (4) 消费者 C 再执行1次循环， $counter==BUFFER_SIZE-2$ ， P_2 不能被唤醒

问题：怎么办？

3.2 信号量机制

■不只是等待信号、发信号？对应睡眠和唤醒！

■还应该能记录一些信息

能记录有“2个进程等待”就可以了...

(1) 缓冲区满， P_1 执行， P_1 sleep，记录下1个进程等待

(2) P_2 执行， P_2 sleep，记录下2个进程等待

(3) C执行1次循环，发现2个进程等待，wakeup 1个

(4) C再执行1次循环，发现？个进程等待，再？

(5) C再执行1次循环，怎么办？此时再来 P_3 怎么办？

什么是信号量？记录一些信息(量)，并根据这个信息决定睡眠还是唤醒(信号)。

信号量开始工作...

初始时 $\text{sem} = ?$

■ (1) 缓冲区满, P_1 执行, $P_1 \text{ sleep}$

$\text{sem} = -1$

什么含义?

■ (2) P_2 执行, $P_2 \text{ sleep}$

$\text{sem} = -2$

■ (3) C 执行 1 次循环, $\text{wakeup } P_1$

$\text{sem} = -1$

■ (4) C 再执行 1 次循环, $\text{wakeup } P_2$

$\text{sem} = 0$

什么含义?

■ (5) C 再执行 1 次循环,

$\text{sem} = 1$

■ (6) P_3 执行, P_2 继续执行

$\text{sem} = 0$

■ 总结一下: 这个整数什么时候-1? 什么时候+1? 什么时候睡眠? 什么时候唤醒?

■ 生产者的信号量, 资源是空格子的个数; 消费者的信号量, 资源是产品的个数

单选题 1分

一种资源的数量是8，这个资源对应的信号量的当前值是2，说明:()

- A** 有2个进程等待这个资源
- B** 有2个资源可以使用
- C** 有6个进程等待这个资源
- D** 有6个资源可以使用

什么是信号量？信号量的定义...

- 信号量: 1965年, 由荷兰学者Dijkstra提出的一种特殊整型变量, 量用来记录, 信号用来sleep和wakeup

```
struct semaphore
{
    int value; //记录资源个数
    PCB *queue;
    //记录等待在该信号量上的进程
}
P(semaphore s); //消费资源
V(semaphore s); //产生资源
```

```
P(semaphore s)
{
    s.value--;
    if(s.value < 0) {
        sleep(s.queue);
    }
}
```

问题: 写出V(s)的代码?

P的名称来源于荷兰语的proberen, 即test

V的名称也来源于荷兰语verhogen(increment)

主观题 10分

参考P(s)，写出V(s)的代码

什么是信号量？信号量的定义...

- 信号量: 1965年, 由荷兰学者Dijkstra提出的一种特殊整型变量, 量用来记录, 信号用来sleep和wakeup

```
struct semaphore
{
    int value; //记录资源个数
    PCB *queue;
    //记录等待在该信号量上的进程
}
P(semaphore s); //消费资源
V(semaphore s); //产生资源
```

P的名称来源于荷兰语的proberen, 即test
V的名称也来源于荷兰语verhogen(increment)

```
P(semaphore s)
{
    s.value--;
    if(s.value < 0) {
        sleep(s.queue);
    }
}
```

问题: 写出V(s)的代码?

```
V(semaphore s)
{
    s.value++;
    if(s.value <= 0) {
        wake(s.queue);
    }
}
```

单选题 1分

当一进程因在互斥信号量mutex上执行V(mutex) 操作而导致唤醒另一个阻塞进程时，则执行V(mutex) 之后mutex.value的值为_____。

- ☐ A 大于0
- ☐ B 小于0
- ☐ C 大于等于0
- ☐ D 小于等于0

用信号量解生产者-消费者问题

```
int fd = open("buffer.txt");  
write(fd, 0, sizeof(int)); //写in  
write(fd, 0, sizeof(int)); //写out
```

用文件定义共享缓冲区

```
semaphore full = 0;  
semaphore empty = BUFFER_SIZE;  
semaphore mutex = 1;
```

信号量的定义和初始化

```
Producer(item) {  
    P(empty); (消费空间? empty--, <0停)  
    P(mutex); 锁门, mutex--为0, 其它进不来  
    读入in; 将item写入到  
in的位置上; 放纸  
    V(mutex); mutex++, 开门  
    V(full); (产生纸巾full++)  
}
```

```
Consumer() {  
    P(full); (消费纸巾full--)  
    P(mutex); 锁门, mutex--  
    读入out; 从文件中的out位置读  
出到item; 打印item; 用纸  
    V(mutex); 开门, mutex++  
    V(empty); (产生空间empty++)  
}
```

单选题 1分

在9个生产者、6个消费者共享8个单元缓冲区的生产者—消费者问题中，互斥使用缓冲区的信号量其初始值为_____。

- ☐ A 1
- ☐ B 6
- ☐ C 8
- ☐ D 9

3.2 信号量机制

- 信号量

- 在操作系统中，信号量代表了一类物理资源，它是相应物理资源的抽象。具体实现时，信号量被定义成具有某种类型的变量。
- 信号量可分为整型信号量和结构体信号量。

1. 整型信号量

- 若信号量为S，则P操作原语和V操作原语可以分别描述如下：
 - `int S;`
 - `P(S): while(S<=0);`
 - `S=S-1;`
 - `V(S): S=S+1;`

3.2 信号量机制

- 2. 结构体型信号量
- 结构体型信号量被定义成具有两个分量成员的结构体类型数据结构。结构体型信号量中的一个分量成员是一个整型变量，它代表当前相应资源的可用数量；另一个分量成员是一个队列指针，指向因等待同类资源而阻塞的进程队列。结构体型信号量可描述如下：

```
typedef struct
{
    int value;    //初值是一个非负整数，表示系统中某类资源的数量
    struct PCB *L;
    // struct PCB为PCB对应的结构体类型，L为指向阻塞队列的指针
}Semaphore;
Semaphore S;
```
- S.value的值：
 - (1) 当S.value > 0时，S.value表示该类资源当前的可用数量。
 - (2) 当S.value = 0时，S.value表示该类资源为空。
 - (3) 当S.value < 0时，S.value的绝对值表示因等待该类资源而阻塞的进程数量。
- 若S.value的初值为1，则表示只有一个临界资源，一段时间内只允许一个进程对该资源进行访问。

结构体型信号量 3.2 信号量机制

- 结构体型信号量S的P(S) 原语操作描述如下:
- void P(Semaphore S)
- {
- lock out interrupts; //关中断
- S.value=S.value-1;
- if(S.value<0) //已无资源S可以分配给运行进程i
- {
- i.status="block"; //置运行进程i的状态为“阻塞”
- Insert(BlockQueue, i); //将进程i插入到阻塞队列S.L中
- unlock interrupts; //开中断
- Scheduler(); //执行进程调度程序调度另一就绪进程运行
- }
- else
- unlock interrupts; //开中断
- }

结构体型信号量

- V(S) 原语操作描述如下:
- void V(Semaphore S)
- {
- lock out interrupts; //关中断
- S.value=S.value+1;
- if (S.value<=0) //阻塞队列中有申请资源S的阻塞进程
- {
- Remove(i); //从S.L所指的阻塞队列中移出队首进程i
- i.status="ready"; //置进程i的状态为“就绪”
- Insert(ReadyQueue, i); //将进程i插入到就绪队列中
- }
- unlock interrupts; //开中断
- }

单选题 1分

设与某互斥资源相关联的信号初值为3，当前值为1，若M表示该资源的可用个数，N表示等待该资源的进程数，则M、N分别是_____。

- ☐ A 0, 1
- ☐ B 1, 0
- ☐ C 1, 2
- ☐ D 2, 0

单选题 1分

用P、V操作实现进程同步，信号量的初值为____

- ☐ A 0
- ☐ B 1
- ☐ C -1
- ☐ D 任意值

单选题 1分

进程从运行状态转变为阻塞状态可能是由于_____。

- ☐ A 进程调度程序的调度
- ☐ B 现运行进程的时间片用完
- ☐ C 现运行进程执行了P操作
- ☐ D 现运行进程执行了V操作

3.3 信号量临界区保护

共同修改信号量引出的问题

初始情况

`empty = -1;`

这是什么含义?

```
Producer(item) {  
    P(empty);  
    ...}
```

一个可能的执行(调度)

```
P1.register = empty;  
P1.register = P1.register - 1;  
P2.register = empty;  
P2.register = P2.register - 1;  
empty = P1.register;  
empty = P2.register;
```

生产者 P_1

```
register = empty;  
register = register - 1;  
empty = register;
```

生产者 P_2

```
register = empty;  
register = register - 1;  
Empty = register;
```

最终的empty等于多少?对吗?

竞争条件(Race Condition)

■ **竞争条件:** 和调度有关的共享数据语义错误

第*i*次执行

```
P1.register = empty;  
P1.register = P1.register - 1;  
P2.register = empty;  
P2.register = P2.register - 1;  
empty = P1.register;  
empty = P2.register;
```

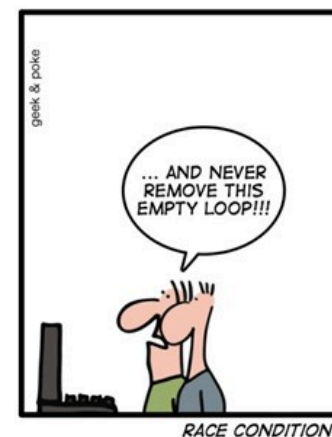
第*j*次执行

```
P1.register = empty;  
P1.register = P1.register - 1;  
empty = P1.register;  
P2.register = empty;  
P2.register = P2.register - 1;  
empty = P2.register;
```

错误由多个进程并发操作共享数据引起

错误和调度顺序有关，难于发现和调试

SIMPLY EXPLAINED



解决竞争条件的直观想法

- 在写共享变量 **empty** 时阻止其他进程也访问 **empty**

仍是那个执行序列

```
P1.register = empty;  
P1.register = P1.register - 1;  
P2.register = empty;  
P2.register = P2.register - 1;  
empty = P1.register;  
empty = P2.register;
```

生产者 **P₁**

检查并给 **empty** 上锁

P₁.register = empty;

P₁.register = P₁.register - 1;

生产者 **P₂**

检查 **empty** 的锁 ?

生产者 **P₁**

empty = P₁.register;

给 **empty** 开锁

生产者 **P₂**

检查并给 **empty** 上锁

P₂.register = empty;

P₂.register = P₂.register - 1;

empty = C.register;

给 **empty** 开锁

一段代码一次只允许
一个进程进入

临界区(Critical Section)

■ **临界区**: 一次只允许一个进程进入的该进程的那一段代码

生产者 P_1

`empty--;`

生产者 P_2

`empty--;`

这就是临界区

■ 一个非常重要的工作: 找出进程中的临界区代码

读写信号量的代码一定是临界区...



这是我们要解决的核心

3.3 信号量临界区保护

- **临界资源与临界区**

- **临界资源**——某段时间内只能允许一个进程使用的资源（即互斥资源）。

- 某段时间内只能允许一个进程使用的资源（即互斥资源）

- **临界区**——进程中访问临界资源的代码段。

- 例：进程P1和进程P2共享一个公用变量S，假设进程P1需要对变量S进行加1操作，而进程P2需要对变量S进行减1操作。

进程P1:	进程P2:
① register1=s;	④ register2=s;
② register1=register1+1;	⑤ register2=register2-1;
③ s=register1;	⑥ s=register2;

- 假设s的当前值是1,

- (1) 先执行语句 ①、②、③，再执行语句 ④、⑤、⑥，则s = 1；

- (2) 先执行语句 ④、⑤、⑥，再执行语句 ①、②、③，则s = 1；

- (3) 执行语句的次序为 ①、②、④、⑤、③、⑥，则s = 0；

-

解决办法：把公用变量s作为临界资源处理，进程P1和进程地P2互斥地访问公用变量s。

临界区代码的保护原则

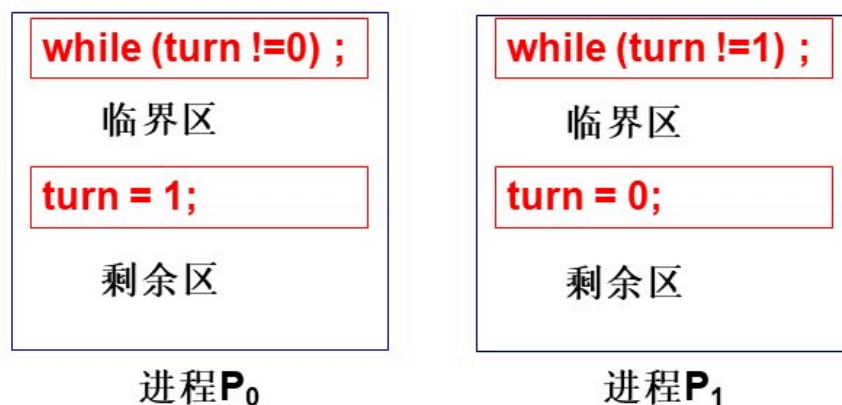
■ **基本原则：互斥进入**：如果一个进程在临界区中执行，则其他进程不允许进入

- 这些进程间的约束关系称为**互斥(mutual exclusion)**
- **这保证了是临界区**

■ **好的临界区保护原则**

- **2. 有空让进**：若干进程要求进入空闲临界区时，应尽快使一进程进入临界区
- **3. 有限等待**：从进程发出进入请求到允许进入，不能无限等待

进入临界区的一个尝试 – 轮换法

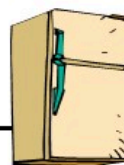


满足互斥进入要求

- 问题: P_0 完成后不能接着再次进入, P_1 阻塞, 尽管进程 P_1 不在临界区...(不满足有空让进)

进入临界区的又一个尝试

■ 似乎没有任何头绪... 可借鉴生活中的道理



时间	丈夫	妻子
3:00	打开冰箱，没有牛奶了	
3:05	离开家去商店	
3:10	到达商店	打开冰箱，没有牛奶了
3:15	买牛奶	离开家去商店
3:20	回到家里，牛奶放进冰箱	到达商店
3:25		买牛奶
3:30		回到家里，牛奶放进冰箱

- 上面的轮换法类似于什么？值日
 - 更好的方法应该是立即去买，留一个便条
- 许多复杂的道理往往就埋藏在日常生活中！



进入临界区的又一个尝试 – 标记法

```
if (noMilk) {  
    if (noNote) {  
        leave Note;  
        buy milk;  
        remove note;  
    }  
}
```

计算机考虑
问题的方式



```
leave Note;  
if (noMilk) {  
    if (noNote) {  
        buy milk;  
    }  
}  
remove note;
```



```
flag[0] = true;  
while (flag[1]);
```

临界区

```
flag[0] = false;
```

剩余区

进程 P_0

```
flag[1] = true;  
while (flag[0]);
```

临界区

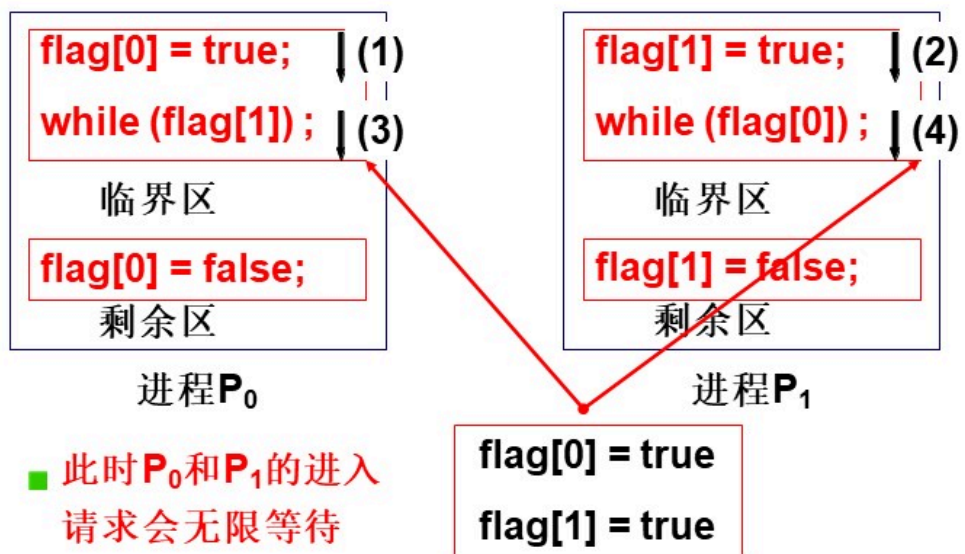
```
flag[1] = false;
```

剩余区

进程 P_1

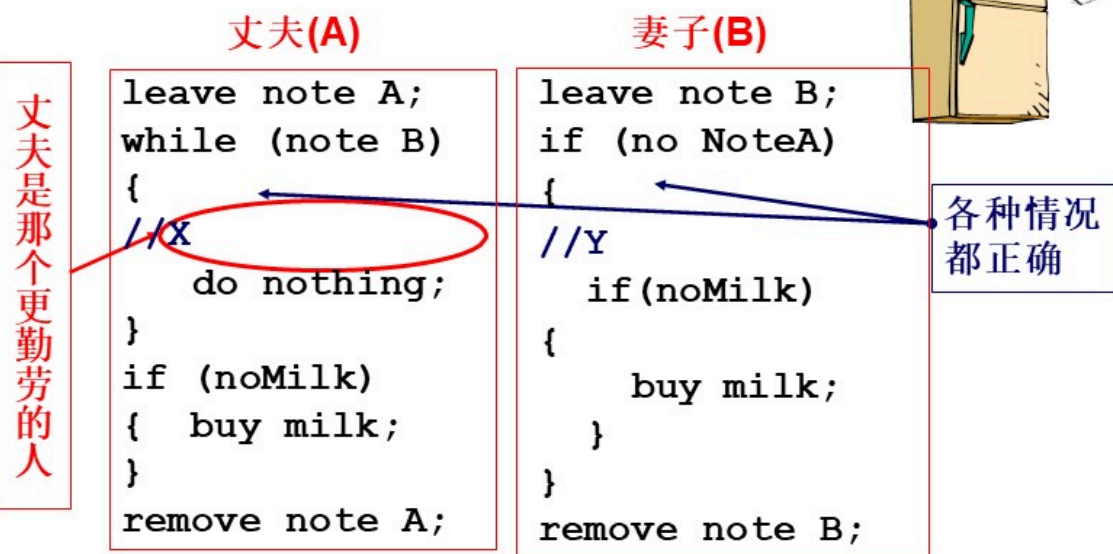
标记法能否解决问题？

■ 考虑下面的执行顺序



进入临界区的再一次尝试 – 非对称标记

■ 带名字的便条 + 让一个人更加勤劳



■ 关键: 选择一个进程进入, 另一个进程循环等待

进入临界区Peterson算法

■ 结合了标记和轮转两种思想

```
flag[0] = true;  
turn = 1;  
while (flag[1] && turn == 1) ;
```

临界区

```
flag[0] = false;
```

剩余区

进程 P_0

```
flag[1] = true;  
turn = 0;  
while (flag[0] && turn == 0) ;
```

临界区

```
flag[1] = false;
```

剩余区

进程 P_1

填空题 3分

进入临界区**Peterson**算法的正确性：

是否满足互斥进入？：[填空1]

是否满足有空让进？：[填空2]

是否满足有限等待？：[填空3]

Peterson算法的正确性

- 满足互斥进入:

如果两个进程都进入, 则

flag[0]=flag[1]=true, turn==0==1, 矛盾!

- 满足有空让进:

- 如果进程 P_1 不在临界区, 则 **flag[1]=false**, 或者**turn=0**, P_0 都能进入!

- 满足有限等待:

P_0 要求进入, **flag[0]=true**; 后面的 P_1 不可能一直进入, 因为 P_1 执行一次就会让**turn=0**。

```
flag[i] = true;  
turn = j;  
while (flag[j] && turn == j);
```

临界区

```
flag[i] = false;
```

剩余区

进程 P_i

多个进程怎么办？– 面包店算法

■ 仍然是标记和轮转的结合

- **如何轮转**: 每个进程都获得一个序号，序号最小的进入
- **如何标记**: 进程离开时序号为0，不为0的序号即标记
- **面包店**: 每个进入商店的客户都获得一个号码，号码最小的先得到服务；号码相同时，名字靠前的先服务。

进程
 P_i

```
choosing[i] = true; num[i] = max(num[0], ..., num[n-1])+1;
choosing[i] = false; for(j=0; j<n; j++)
{
    while(choosing[j]); 有人取号时等待
    while ((num[j] != 0) && (num[j], j)<(num[i], i)); 前面的号不为0，而且号比我小
}
```

临界区

```
num[i] = 0;
```

剩余区

面包店算法的正确性

进程
 P_i

```
choosing[i] = true; num[i] = max(num[0], ..., num[n-1])+1;  
choosing[i] = false; for(j=0; j<n; j++) { while(choosing[j]);  
while ((num[j] != 0) && (num[j], j)<(num[i], i)); }
```

临界区

```
num[i] = 0;
```

- **互斥进入**: P_i 在临界区内, P_k 试图进入, 一定有 $(num[i], i) < (num[k], k)$, P_k 循环等待。
- **有空让进**: 如果没有进程在临界区中, 最小序号的进程一定能够进入。
- **有限等待**: 离开临界区的进程再次进入一定排在最后(FIFO), 所以任一个想进入进程至多等 n 个进程

一个显然的感觉是: 太复杂了, 有没有简单一些的...

• 进入临界区的软件方法

- 1. 两标志进程互斥算法
- 基本思想是：为希望访问临界资源的两个并发进程设置的两个标志T1和T2来表示某个进程是否在临界区；若 T_i ($i=1, 2$) 等于0则表示进程 P_i ($i=1, 2$) 没有在临界区，若 T_i ($i=1, 2$) 等于1则表示进程 P_i ($i=1, 2$) 在临界区。每个进程在进入临界区之前，先判断临界区是否已被另一进程访问，若是则本进程等待，否则本进程进入临界区。

•2. 三标志进程互斥算法

•基本思想是：设置3个标志T1、T2和T，其中T1和T2的作用与两标志进程互斥算法相同，而T是一个公共标志，用来表示允许进入临界区的进程标号；若进程希望进入临界区，则先设置自己的标志Ti (i=1, 2)，然后再检测公共标志T，若T等于i (i=1, 2)，则表示允许进程Pi (i=1, 2) 进入临界区。

进程P1:	进程P2:
...	...
T1=1; //设置P1在临界区标志	T2=1; //设置P2在临界区标志
T=2; //允许P2进入临界区	T=1; //允许P1进入临界区
while(T2==1&&T==2);	while(T1==1&&T==1);
临界区;	临界区;
T1=0; //设置P1不在临界区标志	T2=0; //设置P2不在临界区标志
...	...

临界区保护的另一类解法...

■再想一下临界区：**只允许一个进程进入**，进入另一个进程意味着什么？

■ **被调度**：另一个进程只有被调度才能执行，才可能进入临界区，如何阻止调度？

关中断



进程 P_i

■ 什么时候不好使？

■ 多CPU(多核)...

问题：为什么不好使？

临界区保护的硬件原子指令法

```
boolean  
  TestAndSet (boolean &x)  
{  
    boolean rv = x;  
    x = true;  
    return rv;  
}
```

一次
执行
完毕

```
while(TestAndSet(&  
lock));
```

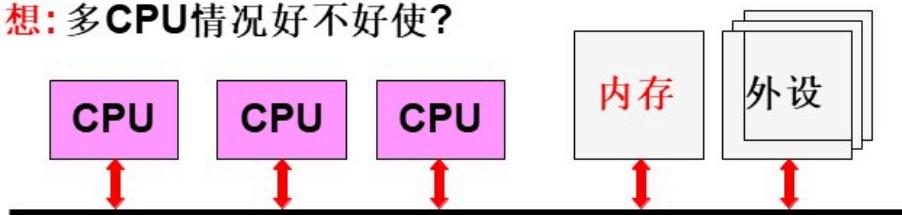
临界区

```
lock = false;
```

剩余区

进程 P_i

■ 想一想:多CPU情况好不好使?



- 实现进程互斥的硬件方法

- 通过计算机提供的一些**机器指令**来实现进程的互斥。
- **机器指令**是指在一个指令周期内执行完成的指令，而专用机器指令的执行则不会被中断。
- **专用机器指令**有3个：
 1. 开关中断指令：进程在进入临界区之前先执行“关中断”指令来屏蔽掉所有中断；进程完成临界区的任务后，再执行“开中断”指令将中断打开。程序结构如下：

```
•   cobegin    //伪代码cobegin和coend表示其间的进程可以并发执行
•   process Pi()      // i=1, 2, 3, ..., n
•   {
•       ...           //与临界资源无关的代码
•       lock out interrupts; //关中断
•       临界区;
•       unlock interrupts; //开中断
•       ...           //与临界资源无关的剩余代码
•   }
•   coend
```

- 2. 测试与设置指令TS (Test and Set)：要为每个临界资源设置一个整型变量s, 可以将它看成一把锁。若s的值为0 (开锁状态)，则表示没有进程访问该锁对应的临界资源。若s的值为1 (关锁状态)，则表示该锁对应的临界资源已被某个进程占用。程序结构如下：

- - `int TS(int s)`
 - `{`
 - `if(s)`
 - `return 1;`
 - `else`
 - `{ s=1;`
 - `return 0;`
 - `}`
 - `}`

- 3. 交换指令（Swap）：要为每个临界资源设置一个整型的全局变量s；若s的值为0则表示没有进程在临界区，若s的值为1则表示有进程在临界区（即访问临界资源）。此外，还要为每个进程设置一个整型局部变量key。只有当s的值为0并且key的值为1时，本进程才能进入临界区。进入临界区后，s的值为1且key的值为0；退出临界区时，应将s的值置为0。程序结构如下：

```
•   int s=0;           // s为全局变量
•
•       cobegin
•   process Pi()        // i=1, 2, 3, ..., n
•   { ...               // 与临界资源无关的代码
•       int key=1;      // key为局部变量
•
•       do{             // 进入区
•
•           Swap(&s, &key);
•       }while(key);
•
•       临界区;
•
•           s=0;         // 退出区
•
•       ...             // 与临界资源无关的剩余代码
•   }
•   coend
```

单选题 1分

对两个并发进程，设互斥信号量为mutex（mutex.value初值为1），若mutex.value当前值为-1，则_____。

- ☐ A 表示没有进程进入临界区
- ☐ B 表示有一个进程进入临界区
- ☐ C 表示有一个进程进入临界区，而另一个进程等待进入临界区
- ☐ D 表示有两个进程进入临界区

单选题 1分

一个正在访问临界资源的进程由于又申请I/O操作而被阻塞时，
_____。

- ☐ A 可以允许其他进程进入该进程的临界区
- ☐ B 不允许其他进程进入临界区和占用CPU执行
- ☐ C 可以允许其他就绪进程占用CPU执行
- ☐ D 不允许其他进程占用CPU执行

单选题 1分

设有 n 个进程共用同一个的程序段（即共享该段），如果每次最多允许 m 个进程（ $m \leq n$ ）同时进入临界区，则等待进入临界区的进程最多可为_____。

- ☐ A n
- ☐ B m
- ☐ C $n-m$
- ☐ D $-m$

3.4 死锁处理

用信号量解生产者-消费者问题

```
int fd = open("buffer.txt");  
write(fd, 0, sizeof(int)); //写in  
write(fd, 0, sizeof(int)); //写out
```

用文件定义共享缓冲区

```
semaphore full = 0;  
semaphore empty = BUFFER_SIZE;  
semaphore mutex = 1;
```

信号量的定义和初始化

```
Producer(item) {  
    P(empty); (empty--纸盒是否满了? <0停)  
    P(mutex); 锁门, mutex--为0, 互斥  
    读入in; 将item写入到  
in的位置上; 放纸  
    V(mutex); mutex++, 开门  
    V(full); (纸巾full++)  
}
```

```
Consumer() {  
    P(full); (full--纸巾是否用完了?)  
    P(mutex); 锁门, mutex--  
    读入out; 从文件中的out位置读  
出到item; 打印item; 用纸  
    V(mutex); 开门, mutex++  
    V(empty); (空间empty++)  
}
```

如果信号量这样使用

```
Producer(item) {  
    P(empty);    mutex  
    P(mutex); empty  
    读入in;将item写入到  
in的位置上;  
    V(mutex);  
    V(full); }  
}
```

```
Consumer() {  
    P(full);    mutex  
    P(mutex); full  
    读入out;从文件中的out  
位置读出到item;打印item;  
    V(mutex);  
    V(empty); }  
}
```

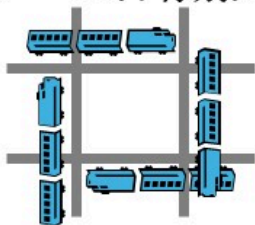
问题：会出什么问题，举一个出现这种情况的例子？

- 我们将这种多个进程由于互相等待对方持有的资源而造成的谁都无法执行的情况叫死锁（银行自助机锁门，取钱，放钱）

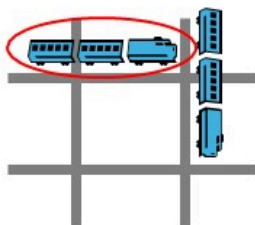
问题：死锁会造成什么结果，为什么？

死锁的成因

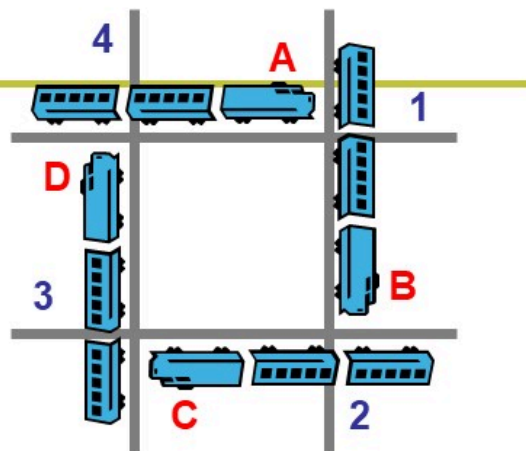
- 资源互斥使用，一旦占有别人无法使用



- 进程占有了一些资源，又不释放，再去申请其他资源



- 各自占有的资源和互相申请的资源形成了环路等待



死锁的4个必要条件

■ 互斥使用(Mutual exclusion)

- 资源的固有特性，如道口

■ 不可抢占(No preemption)

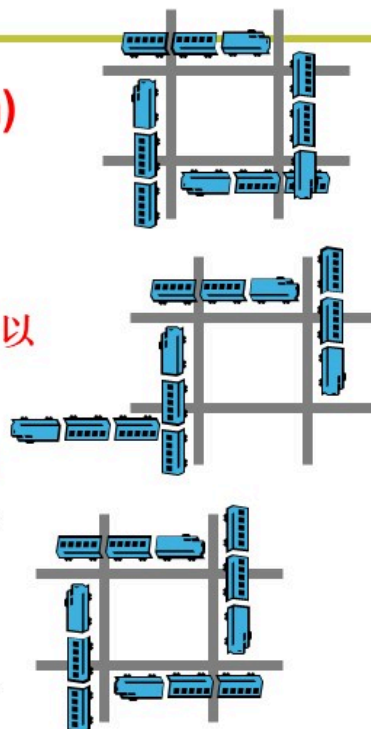
- 资源只能自愿放弃，如车开走以后

■ 请求和保持(Hold and wait)

- 进程必须占有资源，再去申请

■ 循环等待(Circular wait)

- 在资源分配图中存在一个环路



单选题 1分

为多道程序提供的可共享资源不足时可能出现死锁。但是，不当的_____也可能产生死锁。

- ☐ A 进程优先权
- ☐ B 资源的顺序分配
- ☐ C 进程推进顺序
- ☐ D 资源分配队列优先权

死锁处理方法概述



■ 死锁预防 “no smoking”，预防火灾

- 破坏死锁出现的条件

■ 死锁避免 检测到煤气超标时，自动切断电源

- 检测每个资源请求，如果造成死锁就拒绝

■ 死锁检测+恢复 发现火灾时，立刻拿起灭火器

- 检测到死锁出现时，让一些进程回滚，让出资源

■ 死锁忽略 在太阳上可以对火灾全然不顾

- 就好像没有出现死锁一样

死锁预防的方法例子

- 死锁预防 (Deadlock Prevention) :破坏死锁的必要条件来控制死锁的策略。
- 1. 破坏“请求和保持”条件
- 要破坏“请求和保持”条件, 可以采用资源预分配策略, 即每个进程在运行之前一次性申请它所需要的全部资源, 并在资源未得到满足之前不投入运行。进程一旦投入运行, 则分配给它的资源就一起归该进程所有且不再提出新的资源请求。
- 优点: 安全。
- 缺点: ①需要预知未来, 编程困难; ② 进程只有获得了全部资源后才能运行, 会导致一些进程长时间得不到运行; ③系统资源严重浪费

- 死锁的预防

- 2. 破坏“不可抢占”条件
- 采用抢占资源分配策略：资源申请必须按序进行。进程在运行过程中根据需要逐个提出资源请求，当一个已经占有了某些资源的进程又提出新的资源请求而未得到满足时，则必须释放它已获得全部资源而进入阻塞状态，待以后需要时再重新申请。
- 缺点：
 - ① 有些资源被抢占后可能会引起错误；
 - ② 实现起来比较复杂且代价太大，资源浪费；
 - ③ 可能存在某些进程的资源总是被抢占而造成“饥饿”。

- 死锁的预防

- 3. 破坏“循环等待”条件

- 采用资源有序分配策略：将系统中所有资源进行编号，并规定进程申请资源时必须严格按照资源编号递增（或递减）的顺序进行。
- 优点：系统的资源利用率和吞吐量得到改善。
- 缺点：
 - ① 进程实际使用资源的顺序不一定与编号顺序一致，资源有序分配策略会造成资源浪费；
 - ② 资源不同的编号方法对资源的利用率有重要影响，且很难找到最优的编号方法；
 - ③ 资源的编号必须相对稳定，当系统添加新种类设备后处理起来比较麻烦；
 - ④ 严格的资源分配顺序使用户编程的自主性受到限制。

主观题 10分

小河中铺了一串垫脚石用于过河。河的两岸各有一队人准备过河，试说明什么是过河问题中的死锁？并给出破坏死锁4个必要条件均可以用于过河问题的解法。

- 当小河中铺了一串垫脚石用于过河。河的两岸各有一队人准备过河，试说明什么是过河问题中的死锁？并给出破坏死锁4个必要条件均可以用于过河问题的解法。
- 垫脚石每次只允许一个人通过，并且两人在河中相遇且都不退让时出现了死锁。破坏死锁4个必要条件实现过河问题的方法如下：
 - （1）破坏互斥条件。将互斥资源改造为共享资源，加宽垫脚石可同时站立两人，即允许两人共享同一块垫脚石。
 - （2）破坏请求和保持（部分分配）条件。在过河前，每个人必须申请使用河中的所有垫脚石（一次性分配全部资源）。
 - （3）破坏不可抢占（不剥夺）条件。当两人在河中相遇时强行要求过河的另一方撤回，即抢占另一方使用的垫脚石资源。
 - （4）破坏循环等待条件。为避免河中两人都要求对方的垫脚石而陷入循环等待，可铺设两串垫脚石供双方过河。

死锁避免: 判断此次请求是否引起死锁?

- 如果系统中的所有进程存在一个可完成的执行序列 $P_1, \dots, P_n$, 则称系统处于 **安全状态**
都能执行完成当然就不死锁
- **安全序列**: 上面的执行序列 $P_1, \dots, P_n$ 如何找?

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P0	0 1 0	7 4 3	2 3 0
P1	3 0 2	0 2 0	
P2	3 0 2	6 0 0	
P3	2 1 1	0 1 0	
P4	0 0 2	4 3 1	

多选题 1分

根据右图，下面哪个是安全序列()?

A P1, P3, P2, P4, P0

B P0, P1, P2, P3, P4

C P3, P0, P1, P2, P4

D P3, P4, P1, P2, P0

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P0	0 1 0	7 4 3	2 3 0
P1	3 0 2	0 2 0	
P2	3 0 2	6 0 0	
P3	2 1 1	0 1 0	
P4	0 0 2	4 3 1	

找安全序列的银行家算法(Dijkstra提出)

```
int Available[1..m]; //每种资源剩余数量
int Allocation[1..n,1..m]; //已分配资源数量
int Need[1..n,1..m]; //进程还需的各种资源数量
int Work[1..m]; //工作向量
bool Finish [1..n]; //进程是否结束
```

```
Work = Available; Finish[1..n] = false;
while(true) {
    for(i=1; i<=n; i++){
        if(Finish[i]==false && Need[i]≤Work) {
            Work = Work + Allocation[i];
            Finish[i] = true; break;}
        else {goto end;}
    }
}
End: for(i=1;i<=n;i++)
    if(Finish[i]==false) return "deadlock";
```

$T(n)=O(mn^2)$

填空题 1分

当前状态如下图，根据银行家算法，给出安全序列 [填空1]

■ 当前状态:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
<i>P0</i>	0 1 0	7 4 3	3 3 2
<i>P1</i>	2 0 0	1 2 2	
<i>P2</i>	3 0 2	6 0 0	
<i>P3</i>	2 1 1	0 1 1	
<i>P4</i>	0 0 2	4 3 1	

死锁避免之银行家算法实例

■ 当前状态:

		<u>Allocation</u>	<u>Need</u>	<u>Available</u>
		A B C	A B C	A B C
	Work=[3 3 2]	P0	0 1 0	7 4 3
P ₁	Work=[5 3 2]	P1	2 0 0	3 3 2
P ₃	Work=[7 4 3]	P2	3 0 2	
P ₂	Work=[10 4 5]	P3	2 1 1	
P ₄	Work=[10 4 7]	P4	0 0 2	
P ₀	Work=[10 5 7]			

■ 安全序列是 $\langle P_1, P_3, P_2, P_4, P_0 \rangle$

请求出现时: 首先假装分配, 然后调用银行家算法

■ P_0 申请 (0,2,0)

	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 3 0	7 2 3	2 1 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- 进程 P_0, P_1, P_2, P_3, P_4 一个也没法执行, 死锁进程组
- 此次申请 (0,2,0) 被拒绝

主观题 10分

设系统有3种类型资源{A, B, C}和5个进程{P1, P2, P3, P4, P5}, A资源的数量为17, B资源的数量为5, C资源的数量为20, 在T0时刻系统状态如表3.1所示。

系统采用银行家算法实施死锁避免策略。

- (1) T0时刻是否为安全状态? 若是请给出安全序列, 若不是请给出不安全的理由。
- (2) 在T0时刻若进程P2请求资源 (0, 3, 4), 是否能实施资源分配? 为什么?
- (3) 若进程P4请求资源 (2, 0, 1), 是否能实施资源分配? 为什么?
- (4) 在 (3) 的基础上若进程P1请求资源 (0, 2, 0), 是否实施资源分配? 为什么?

表 3.1 T0 时刻系统状态

进程	最大资源需求量			已分配资源数量		
	A	B	C	A	B	C
P1	5	5	9	2	1	2
P2	5	3	6	4	0	2
P3	4	0	11	4	0	5
P4	4	2	5	2	0	4
P5	4	2	4	3	1	4
剩余资源数	A		B		C	
	2		3		3	

判断系统是否为安全状态，关键在于能否找到一个进程的安全序列，这与进程在T0时刻还需要的资源数量有关，即资源需求量Need=最大资源需求量-已分配资源数量。由此得到表3.2所示。

表 3.2 T0 时刻资源分配情况表

进程	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P1	2	1	2	3	4	7	2	3	3
P2	4	0	2	1	3	4			
P3	4	0	5	0	0	6			
P4	2	0	4	2	2	1			
P5	3	1	4	1	1	0			

(1) 根据表3.2所示，找到一个进程其还需要的资源应能够由系统现剩余的可用资源予以满足。因此，该进程在获得系统资源后可运行至结束，且结束后即将该进程所占用的全部资源归还（加入）到系统的可用资源中。然后，再寻找下一个可以获得系统资源而运行至结束的进程。如果最终可以找到一个使所有进程都能获得系统资源而运行至结束的进程运行序列，则这个序列就称为安全序列。利用银行家算法对T0时刻的资源分配情况进行分析，可以得到T0时刻的安全性分析如表3.3所示。

表 3.3 T0 时刻系统的安全性分析

进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	true
P4	2	3	3	1	1	0	3	1	4	5	4	7	true
P5	5	4	7	2	2	1	2	0	4	7	4	11	true
P3	7	4	11	0	0	6	4	0	5	11	4	16	true
P2	11	4	16	1	3	4	4	0	2	15	4	18	true
P1	15	4	18	3	4	7	2	1	2	17	5	20	true

从表3.3所示可以看出，在T0时刻存在一个安全序列（**P5**, **P4**, P3, P2, P1），故该状态是安全的

（2）在T1时刻因进程P2的请求资源 $(0, 3, 4) >$ 剩余资源 $(2, 3, 3)$ ，所以不能分配。

(3) 若进程P4的请求资源为 (2,0,1)，则按银行家算法进行如下检查：

- ① P4请求资源 (2, 0, 1) ≤ P4资源需求量 (2, 2, 1)。
- ② P4请求资源 (2, 0, 1) ≤ 剩余资源 (2, 3, 3)。
- ③ 此时的资源分配情况如表3.4所示（已分配给P4(2,0,1)之后的情况）。

表 3.4 资源分配情况表

进程	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P1	2	1	2	3	4	7	0	3	2
P2	4	0	2	1	3	4			
P3	4	0	5	0	0	6			
P4	4	0	5	0	2	0			
P5	3	1	4	1	1	0			

此时的安全性分析如表3.5所示从表3.5所示可以看出，
此时刻存在一个安全序列 (P4, P5, P3, P2, P1)，即可以将P4申请的资源分配给它。

表 3.5 系统的安全性分析

进程	Work			Need			Allocation			Work+Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	true
P4	0	3	2	0	2	0	4	0	5	4	3	7	true
P5	4	3	7	1	1	0	3	1	4	7	4	11	true
P3	7	4	11	0	0	6	4	0	5	11	4	16	true
P2	11	4	16	1	3	4	4	0	2	15	4	18	true
P1	15	4	18	3	4	7	2	1	2	17	5	20	true

(4) 在(3)的基础上,若进程P1请求资源(0, 2, 0),则按银行家算法进行如下检查:

- ① P1请求资源(0, 2, 0) ≤ P1资源需求量(3, 4, 7)。
- ② P1请求资源(0, 2, 0) ≤ 剩余资源(0, 3, 2)。
- ③ 此时的资源分配情况如表3.6所示(已分配给P1(0, 2, 0)之后的情况)。

表 3.6 资源分配情况表

进程	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P1	2	3	2	3	2	7	0	1	2
P2	4	0	2	1	3	4			
P3	4	0	5	0	0	6			
P4	4	0	5	0	2	0			
P5	3	1	4	1	1	0			

此时,可用资源Available已不能满足任何进程的资源需求,因此不能将资源分配给P1。

死锁检测+恢复: 发现问题再处理

■ 基本原因: 每次申请都执行 $O(mn^2)$, 效率低。发现问题再处理

■ 定时检测或者是发现资源利用率低时检测

```
Finish[1..n] = false;  
if(Allocation[i] == 0) Finish[i]=true;  
...//和Banker算法完全一样  
for (i=1;i<=n;i++)  
    if (Finish[i]==false)  
        deadlock = deadlock + {i};
```

■ 选择哪些进程回滚? 优先级? 占用资源多的? ...

■ 如何实现回滚? 那些已经修改的文件怎么办?

死锁忽略的引出

- 死锁预防？
 - 引入太多不合理因素...
- 死锁避免？
 - 每次申请都执行银行家算法 $O(mn^2)$ ，效率太低
- 死锁检测+恢复？
 - 恢复很不容易，进程造成的改变很难恢复
- 死锁忽略
 - 死锁出现不是确定的，又可以用重启来处理死锁
 - 有趣的是大多数非专门的操作系统都用它，如
UNIX, Linux, Windows

单选题 1分

某系统中有11台打印机，N个进程共享打印机资源，每个进程要求3台，在N的取值不超过_____时，系统不会发生死锁。

- ☐ A 4
- ☐ B 5
- ☐ C 6
- ☐ D 7

单选题 1分

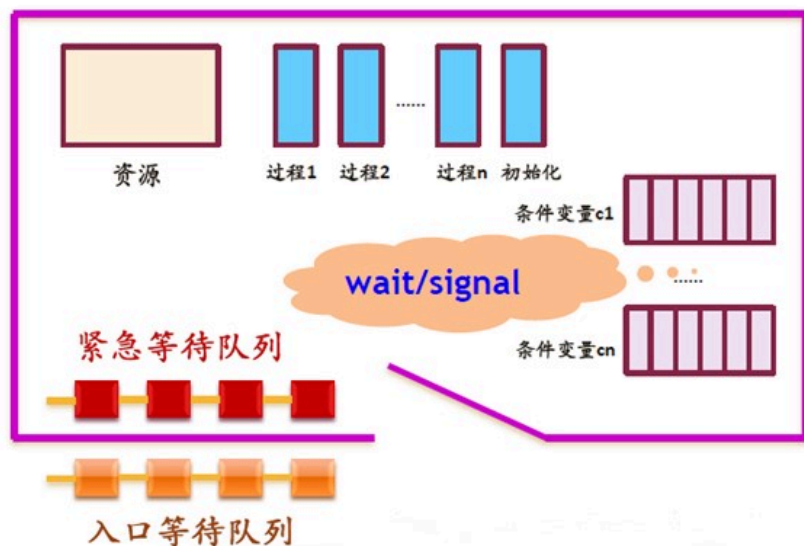
设 m 为同类资源数， n 为系统中并发进程数，当 n 个进程共享 m 个互斥资源时，每个进程的最大需求是 w ，则下列情况会出现死锁的是_____。

- ☐ A $m=2, n=1, w=2$
- ☐ B $m=2, n=2, w=1$
- ☐ C $m=4, n=3, w=2$
- ☐ D $m=4, n=2, w=3$

3.5 管程机制

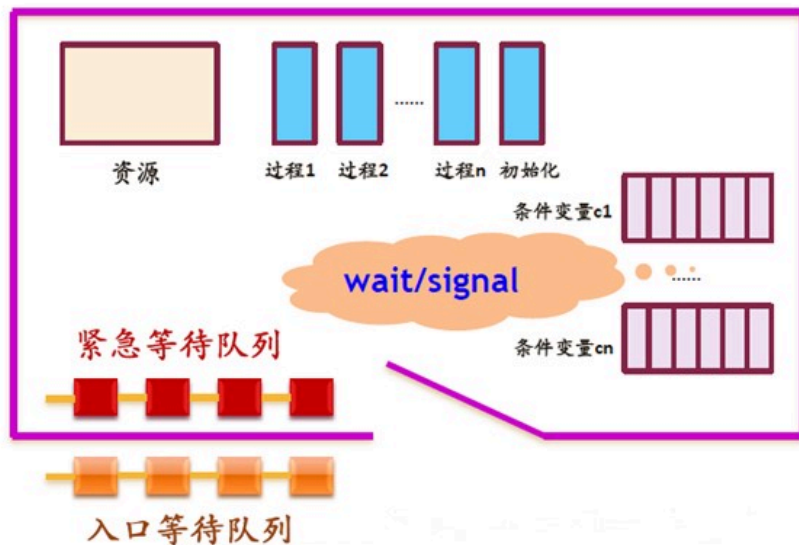
- **动机：为什么有管程？ 传统PV和锁机制有如下问题：**
 - 程序易读性差：要了解对于一组共享变量及信号量的操作是否正确，则必须通读整个系统或者并发程序。
 - 程序不利于修改和维护：程序局部性很差，所以任一组变量或一段代码的修改都可能影响全局。
 - 正确性难以保证：操作系统或并发程序通常很大，很难保证一个复杂的系统没有逻辑错误。
 - 容易发生死锁：如果不使用好P、V操作时，逻辑上发生错误，很有可能会导致死锁。

3.5 管程机制

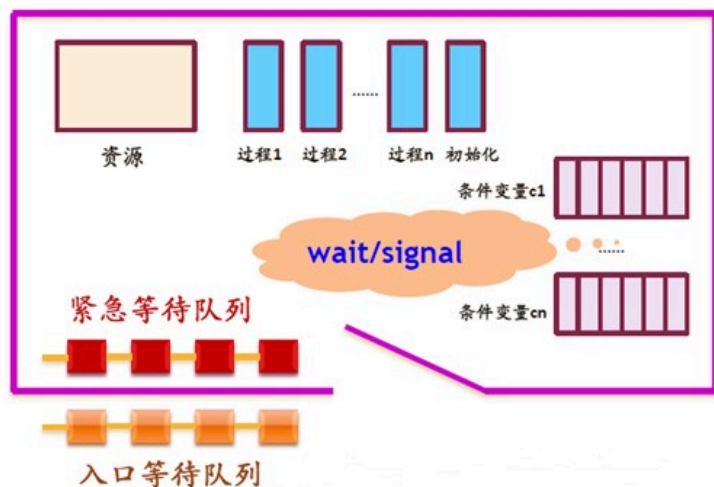


- 管程的组成：一个由过程（函数）、变量及数据结构等组成的一个集合
 - **一个锁**：控制管程代码的互斥访问
 - **0或者多个条件变量**：管理共享数据的并发访问，每个条件变量有个等待（紧急）队列
 - **入口等待队列**：管程入口处等待队列
 - **紧急等待队列**：某个条件变量的等待队列（为资源占用而等待）
 - **条件变量队列**：紧急等待队列优先级高于条件变量等待队列

3.5 管程机制



- 管程-条件变量
- 管程中的共享变量在管程外部是不可见的，外部只能通过调用管程中所说明的外部过程(函数)来间接地访问管程中的共享变量
 - 互斥：任一时刻管程中只能有一个活跃进程，通过锁竞争进入管程
 - 等待：进入管程的线程**因资源被占用**而进入等待状态
 - 每个条件变量表示一种等待原因，对应一个等待队列
 - 入口队列管理未进入管程的线程/进程
 - 唤醒：管程中等待的线程可以（其他线程释放资源）被唤醒
 - 同步：等待和唤醒操作
 - 管程操作：进入enter, 离开leave, 等待wait, 唤醒signal



- 同步：条件变量 (Condition Variables)以及相关的两个操作：wait和signal，处理等待机制。
- Lock
 - Lock Acquire：等待锁可用，然后抢占锁。
 - Lock release：释放锁，如果有等待者则唤醒。
- Wait()操作：释放锁，睡眠，重新获得锁后返回。
 - 将自己阻塞在等待队列中
 - 释放进程的互斥访问
 - 将一个等在进程之外的进程调入
- Signal()操作：如果有睡眠的，唤醒等待者
 - 将等待队列中的一个线程唤醒
 - 如果等待队列为空，则等同空操作

- 管程实现

```
Class Condition {  
    int numWaiting = 0;  
    WaitQueue q;  
}
```

```
Condition::Wait(lock) {  
    numWaiting++;  
    Add this thread t to q;  
    release(lock);  
    schedule(); //need mutex  
    require(lock);  
}
```

```
Condition::Signal() {  
    if (numWaiting > 0) {  
        Remove a thread t from q;  
        wakeup(t); //need mutex  
        numWaiting--;  
    }  
}
```

- 管程在进程同步中的应用

- 1. 生产者-消费者问题的管程解决方法

- 在生产者-消费者问题中，生产者进程生产完一个产品后，如果缓冲池中所有的缓冲区都装满产品，则生产者进程被阻塞；同样，消费者进程需要消费一个产品时，如果此时缓冲池中所有的缓冲区都为空，则消费者进程被阻塞。需要设置两个不同的阻塞队列，即引入两个条件变量：`notfull`和`notempty`来分别对应这两种不同的阻塞条件。

- 管程实现生产者消费者

```
class BoundedBuffer {  
    ...  
    Lock lock;  
    int count = 0;  
    Condition notFull, notEmpty;  
}
```

```
BoundedBuffer::Deposit(c) {
```

```
    Add c to the buffer;  
    count++;
```

```
}
```

```
BoundedBuffer::Remove(c) {
```

```
    Remove c from buffer;  
    count--;
```

```
}
```

- 管程实现生产者消费者

```
classBoundedBuffer {  
    ...  
    Lock lock;  
    int count = 0;  
    Condition notFull, not Empty;  
}
```

```
BoundedBuffer::Deposit(c) {  
    lock->Acquire();  
  
    Add c to the buffer;  
    count++;  
  
    lock->Release();  
}
```

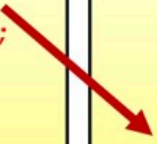
```
BoundedBuffer::Remove(c) {  
    lock->Acquire();  
  
    Remove c from buffer;  
    count--;  
  
    lock->Release();  
}
```

- 管程实现生产者消费者

```
classBoundedBuffer {  
    ...  
    Lock lock;  
    int count = 0;  
    Condition notFull, notEmpty;  
}
```

```
BoundedBuffer::Deposit(c) {  
    lock->Acquire();  
    while (count == n)  
        notFull.Wait(&lock);  
    Add c to the buffer;  
    count++;  
  
    lock->Release();  
}
```

```
BoundedBuffer::Remove(c) {  
    lock->Acquire();  
  
    Remove c from buffer;  
    count--;  
    notFull.Signal();  
    lock->Release();  
}
```



- 管程实现生产者消费者

```
class BoundedBuffer {  
    ...  
    Lock lock;  
    int count = 0;  
    Condition notFull, notEmpty;  
}
```

```
BoundedBuffer::Deposit(c) {  
    lock->Acquire();  
    while (count == n)  
        notFull.Wait(&lock);  
    Add c to the buffer;  
    count++;  
    notEmpty.Signal();  
    lock->Release();  
}
```

```
BoundedBuffer::Remove(c) {  
    lock->Acquire();  
    while (count == 0)  
        notEmpty.Wait(&lock);  
    Remove c from buffer;  
    count--;  
    notFull.Signal();  
    lock->Release();  
}
```

管程中条件变量的释放处理方式

- 如果线程T1因条件A未满足处于阻塞状态，那么当线程T2让条件A满足并执行signal操作唤醒T1后，不允许线程T1和T2同时处于管程中，那么如何确定哪个执行/哪个等待？
- 可采用下面方式之一进行处理：
 - Hoare: T2 通知完 T1后，T2 阻塞，T1 马上执行；等 T1 执行完，再唤醒 T2 执行
 - Hansen: T2 通知完 T1 后，T2 还会接着执行，T2 执行结束后（规定：最后操作是signal），然后 T1 再执行（将锁直接给T1）
 - MESA: T2 通知完 T1 后，T2 还会接着执行，T1 并不会立即执行，而是重新竞争访问权限

管程中条件变量的释放处理方式

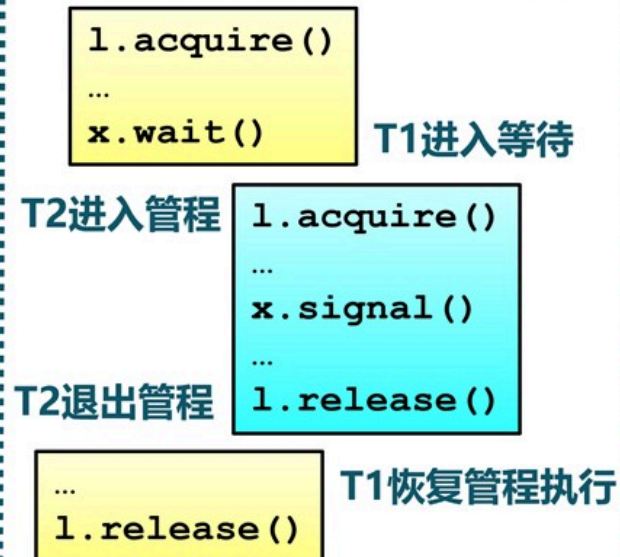
■ Hoare管程

■ 主要见于教材中



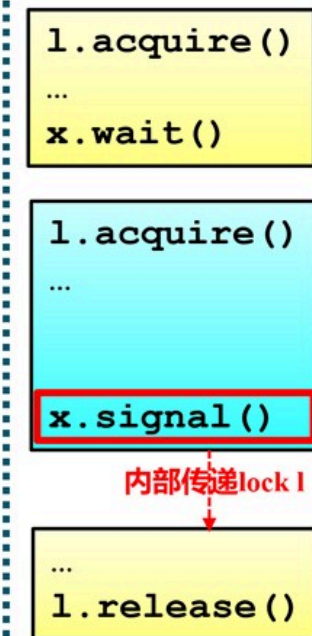
■ MESA管程

■ 主要用于真实OS和Java中



■ Hansen管程

■ Concurrent Pascal



3.5 管程机制

- 条件变量与管程结构

- 1. 条件变量
- 为了避免多个进程在管程中出现“忙等”现象，管程提供了条件变量机制，让暂时不能进入管程的进程阻塞或挂起等待。条件变量 (Condition Variable) 是管程提供的一种进程同步机制，它能够在多个进程之间传递信号并解决管程内进程可能出现的“忙等”问题。
- 条件变量是封装在管程内的一种数据结构，它对应一个进程阻塞队列并只能被管程中的过程（函数）访问，且是管程内的所有过程（函数）的全局变量。条件变量只能通过wait和signal两个原语进行控制，而wait和signal原语总是在某个条件变量（如X）上执行，表示为X.wait或X.signal。X.wait操作的作用是使本进程阻塞在条件变量X上，X.signal操作的作用是若有进程的被阻塞在条件变量X上，则将第一个阻塞进程唤醒。

3.5 管程机制

- 条件变量与信号量的区别：
 - (1) 与信号量具有值的概念不同，条件变量没有关联的值；
 - (2) 在管程中执行signal操作时，如果相应的阻塞队列中没有阻塞进程则本次操作为空操作，即什么也不做；
 - (3) 在信号量中则不会出现这种情况，V操作在阻塞队列中没有阻塞进程时也必须对信号量进行加1操作（资源个数增1）。
- 2. 管程结构
 - 管程的定义：一个管程定义了一个数据结构和在该数据结构上执行的一组过程（函数），这组过程（函数）能够同步进程和改变管程中的局部数据。

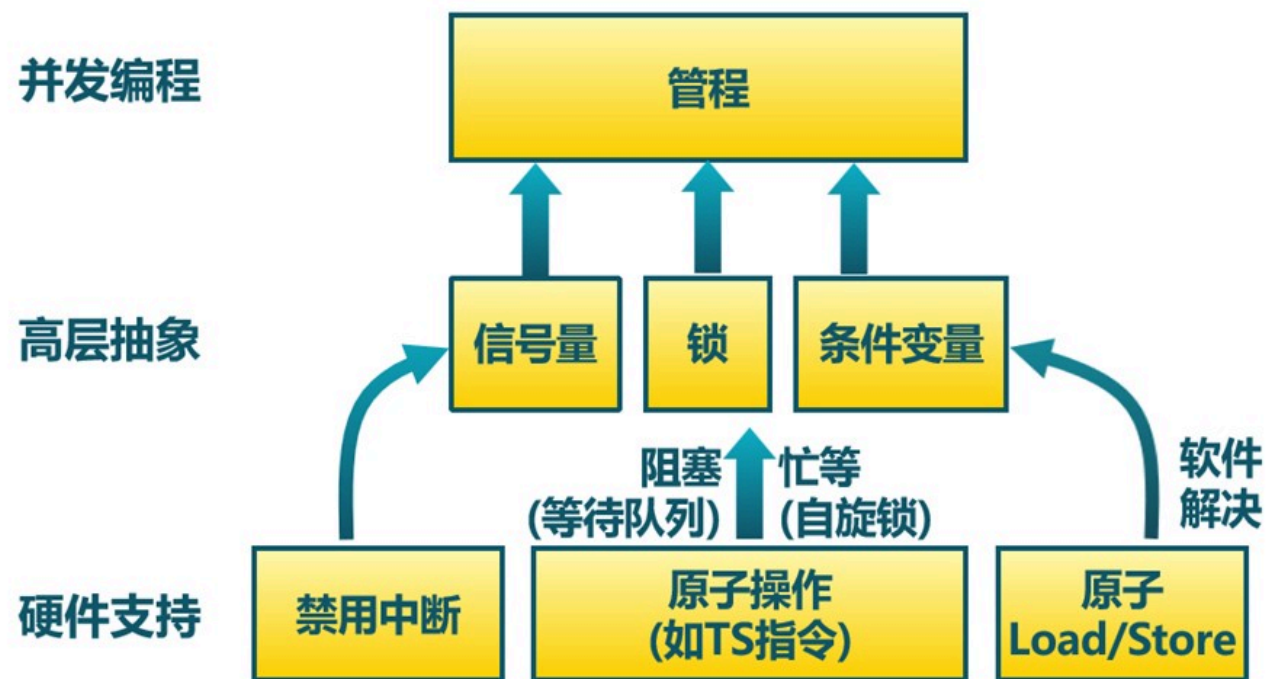
3.5 管程机制

- **管程的性质：**

- (1) 管程中不仅有数据而且有对数据的操作，即管程是一种扩展了的抽象数据类型。
- (2) 管程这种扩展了的抽象数据类型的描述对象是互斥资源，因此，管程实际上是互斥资源的管理模块。
- (3) 作为一个软件模块，管程应符合模块化的要求，即管程应是一个基本程序单位，并可以被单独编译。
- (4) 管程中的数据结构只能被管程中的过程（函数）使用，管程内的数据结构和过程（函数）的具体实现在外部是不可见的。
- (5) 为了实现对互斥资源的共享，管程应有互斥与同步机制。

- 使用管程实现进程同步具有如下特点：
- （1）管程内的数据结构只能由管程定义的过程（函数）访问，任何外部过程（函数）都不能访问它们。
- （2）任何时刻只能有一个进程在管程中执行，其他申请进入管程的进程必须阻塞等待。
- （3）一个外部过程（函数）只有通过调用管程内的一个过程（函数）才能进入管程。

3.4 管程机制



3.4 管程机制

- **管程缺点：**

- (1) 管程对编译器的依赖性。因为管程需要编译器将互斥原语（P、V操作）加在管程的开始和结尾（管程一次仅允许一个进程访问，因而是临界资源），但对许多程序设计语言来说，并没有实现管程的机制。
- (2) 管程只能在单台计算机上发挥效率。由于那些直接支持管程的原语并没有提供计算机之间的信息交换方法，因此管程无法在多CPU计算机环境下或者网络环境下发挥作用。

单选题 1分

下述哪个选项不是管程的组成部分_____。

- ☐ A 局部于管程内的数据结构
- ☐ B 对管程内数据结构进行操作的一组过程（函数）
- ☒ C 管程外过程（函数）调用管程内数据结构的说明
- ☐ D 对管程内数据结构设置的初始化语句

单选题 1分

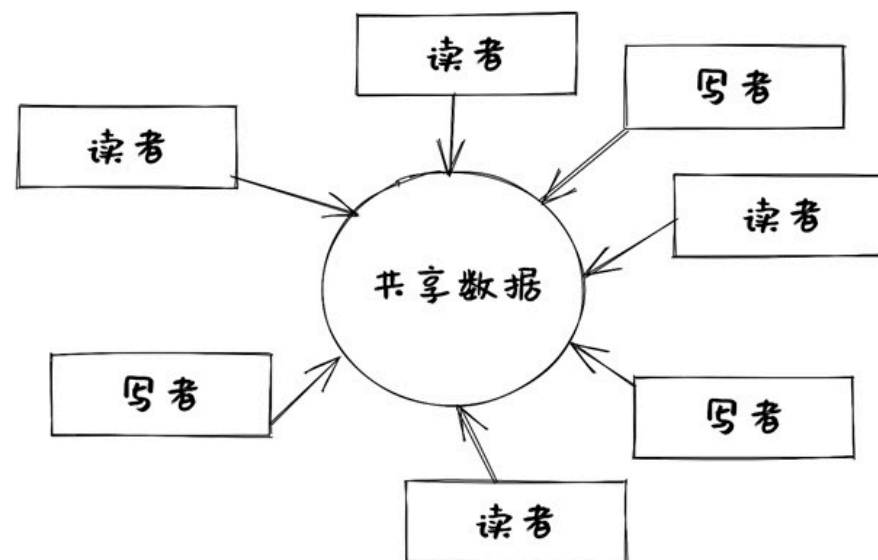
以下关于管程的描述中，错误的是_____。

- ☐ A 管程是进程同步工具，用于解决信号量机制大量同步操作分散的问题
- ☐ B 管程每次只允许一个进程进入管程
- ☒ C 管程中signal操作的作用和信号量机制中的V操作相同
- ☐ D 管程是被进程调用的，管程是语法单位，无创建和撤销

3.5 经典互斥与同步问题

读者-写者问题

- 共享数据的两类使用者
 - 读者：只读不修改数据
 - 写者：读取和修改数据
- 对共享数据的读写
 - 多个：“读—读”——允许
 - 单个：“读—写”——互斥
 - 单个：“写—写”——互斥



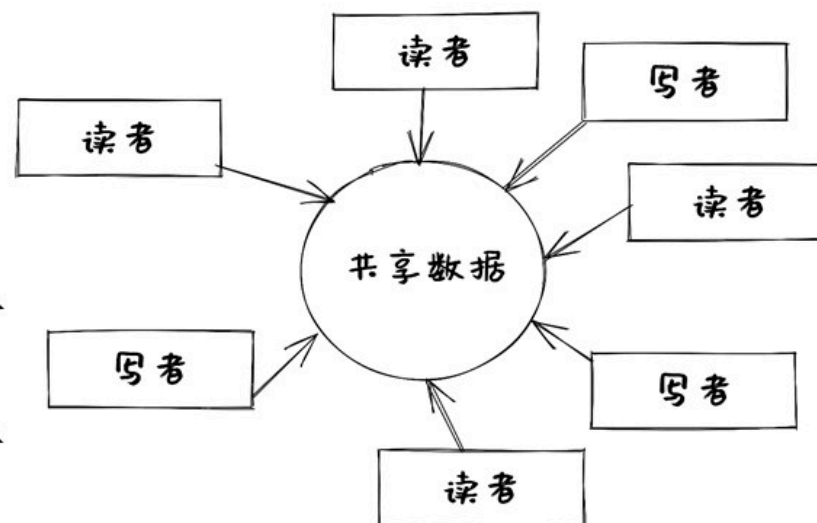
读者-写者问题

• 读者优先策略

- 只要有读者正在读状态，后来的读者都能直接进入
- 如读者持续不断进入，则写者就处于饥饿

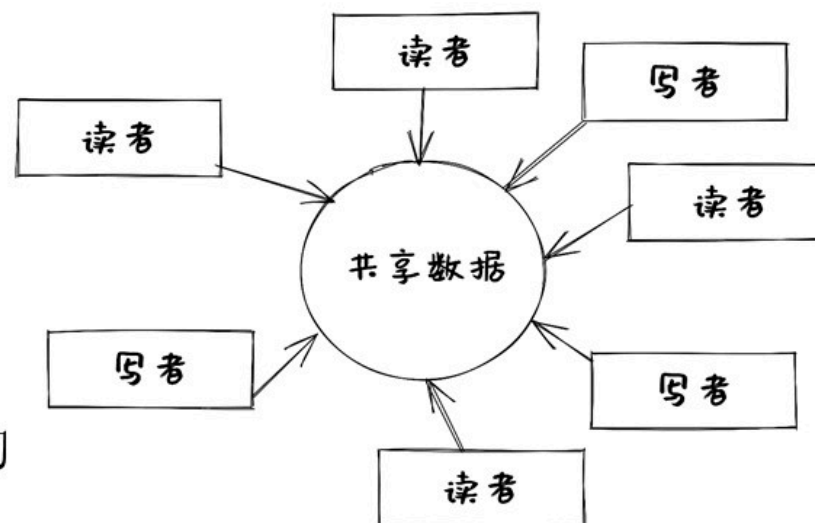
• 写者优先策略

- 只要有写者就绪，写者应尽快执行写操作
- 如写者持续不断就绪，则读者就处于饥饿



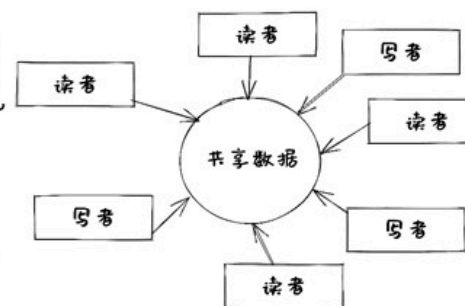
读者-写者问题

- 用信号量描述每个约束
 - 信号量WriteMutex: 控制读写操作的互斥, 初始化为1
 - 读者计数Rcount : 正在进行读操作的读者数目, 初始化为0
 - 信号量CountMutex: 控制对读者计数的互斥修改, 初始化为1



读者-写者问题

- 问题是：有一个数据文件可以被多个进程共享，各进程共享数据文件的方式不同。有的进程只是从文件中读取信息，这类进程称为“读者进程”。有的进程写信息到文件中或者又读又写，这些进程统称为“写者进程”。为了不使文件内容混乱要求各进程在使用文件时必须遵守以下规定：
 - (1) 允许多个读者进程同时读文件（读操作不破坏文件）。
 - (2) 不允许写者进程与其他进程同时访问文件（有写操作存在可能造成读取信息混乱）。
- 读者-写者问题中的进程之间存在**3种制约关系**：一是读者之间允许同时读；二是读者与写者之间需要互斥；三是写者与写者之间也需要互斥。
- 为了解决读者进程、写者进程之间的同步，需要设置如下的信号量：
 - (1) 读互斥信号量`rmutex`。用于使读者进程互斥地访问公用变量（共享变量）`count`，且`rmutex.value`的初值为1。
 - (2) 写互斥信号量`wmutex`。用于实现写者进程与读者进程的互斥以及写者进程与写者进程的互斥，且`wmutex.value`的初值为1。
 - (3) 公用变量`count`。用于记录当前正在读文件的读者进程个数，且`count.value`的初值为0，仅在`count.value`的值为0时才允许写者进程访问文件。



读者-写者问题 读者优先

Writer

`write;`

Reader

`read;`

读者-写者问题 读者优先

Writer

```
P (WriteMutex) ;  
  
write;  
  
V (WriteMutex) ;
```

Reader

```
P (WriteMutex) ;  
  
read;  
  
  
  
V (WriteMutex) ;
```

读者-写者问题 读者优先

Writer

```
P(WriteMutex);  
  
write;  
  
V(WriteMutex);
```

Reader

```
if (Rcount == 0)  
    P(WriteMutex);  
    ++Rcount;  
  
read;  
  
V(WriteMutex);
```

读者-写者问题 读者优先

Writer

```
P(WriteMutex);  
  
write;  
  
V(WriteMutex);
```

Reader

```
if (Rcount == 0)  
    P(WriteMutex);  
++Rcount;  
  
read;  
  
--Rcount;  
if (Rcount == 0)  
    V(WriteMutex);
```

读者-写者问题 读者优先

Writer

```
P(WriteMutex);  
  
write;  
  
V(WriteMutex);
```

Reader

```
P(CountMutex);  
if (Rcount == 0)  
    P(WriteMutex);  
++Rcount;  
V(CountMutex);  
  
read;  
  
--Rcount;  
if (Rcount == 0)  
    V(WriteMutex);
```

读者-写者问题 读者优先

Writer

```
P(WriteMutex);  
  
write;  
  
V(WriteMutex);
```

Reader

```
P(CountMutex);  
if (Rcount == 0)  
    P(WriteMutex);  
++Rcount;  
V(CountMutex);  
  
read;  
  
P(CountMutex);  
--Rcount;  
if (Rcount == 0)  
    V(WriteMutex);  
V(CountMutex)
```

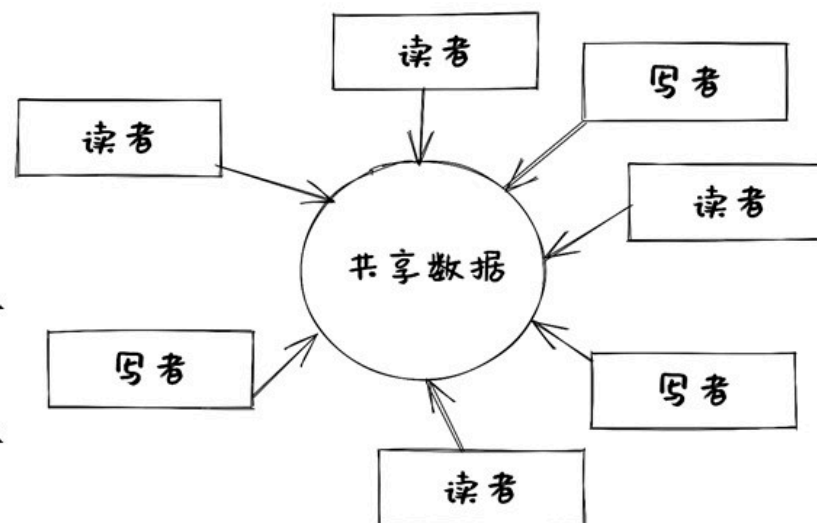
读者-写者问题

•读者优先策略

- 只要有读者正在读状态，后来的读者都能直接进入
- 如读者持续不断进入，则写者就处于饥饿

•写者优先策略

- 只要有写者就绪，写者应尽快执行写操作
- 如写者持续不断就绪，则读者就处于饥饿



管程的状态变量

■ 两个基本方法

```
Database::Read() {  
    Wait until no writers;  
    read database;  
    check out - wake up waiting writers;  
}
```

```
Database::Write() {  
    Wait until no readers/writers;  
    write database;  
    check out - wake up waiting readers/writers;  
}
```

■ 管程的状态变量

```
AR = 0;           // # of active readers  
AW = 0;           // # of active writers  
WR = 0;           // # of waiting readers  
WW = 0;           // # of waiting writers
```

管程的状态变量

■ 两个基本方法

```
Database::Read() {  
    Wait until no writers;  
    read database;  
    check out - wake up waiting writers;  
}
```

```
Database::Write() {  
    Wait until no readers/writers;  
    write database;  
    check out - wake up waiting readers/writers;  
}
```

■ 管程的状态变量

```
AR = 0;           // # of active readers  
AW = 0;           // # of active writers  
WR = 0;           // # of waiting readers  
WW = 0;           // # of waiting writers  
Lock lock;  
Condition okToRead;  
Condition okToWrite;
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();

    lock.Release();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();

    AR++;
    lock.Release();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();
    while (???) {
        WR++;
        okToRead.wait(&lock);
        WR--;
    }
    AR++;
    lock.Release();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();
    while ((AW+WW) > 0) {
        WR++;
        okToRead.wait(&lock);
        WR--;
    }
    AR++;
    lock.Release();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();
    while ((AW+WW) > 0) {
        WR++;
        okToRead.wait(&lock);
        WR--;
    }
    AR++;
    lock.Release();
}
```

```
Private Database::DoneRead() {
    lock.Acquire();
    AR--;

    lock.Release();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();
    while ((AW+WW) > 0) {
        WR++;
        okToRead.wait(&lock);
        WR--;
    }
    AR++;
    lock.Release();
}
```

```
Private Database::DoneRead() {
    lock.Acquire();
    AR--;
    if (???) {
        okToWrite.signal();
    }
    lock.Release();
}
```

读者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Read() {
    //Wait until no writers;
    StartRead();
    read database;
    //check out - wake up waiting writers;
    DoneRead();
}
```

```
Private Database::StartRead() {
    lock.Acquire();
    while ((AW+WW) > 0) {
        WR++;
        okToRead.wait(&lock);
        WR--;
    }
    AR++;
    lock.Release();
}
```

```
Private Database::DoneRead() {
    lock.Acquire();
    AR--;
    if (AR == 0 && WW > 0) {
        okToWrite.signal();
    }
    lock.Release();
}
```

写者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Private Database::StartWrite() {
    lock.Acquire();

    AW++;
    lock.Release();
}
```

```
Public Database::Write() {
    //Wait until no readers/writers;
    StartWrite();
    write database;
    //check out-wake up waiting readers/writers;
    DoneWrite();
}
```

写者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Private Database::StartWrite() {
    lock.Acquire();
    while (???) {
        WW++;
        okToWrite.wait(&lock);
        WW--;
    }
    AW++;
    lock.Release();
}
```

```
Public Database::Write() {
    //Wait until no readers/writers;
    StartWrite();
    write database;
    //check out-wake up waiting readers/writers;
    DoneWrite();
}
```

写者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Private Database::StartWrite() {
    lock.Acquire();
    while ((AW+AR) > 0) {
        WW++;
        okToWrite.wait(&lock);
        WW--;
    }
    AW++;
    lock.Release();
}
```

```
Public Database::Write() {
    //Wait until no readers/writers;
    StartWrite();
    write database;
    //check out-wake up waiting readers/writers;
    DoneWrite();
}
```

写者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Write() {
    //Wait until no readers/writers;
    StartWrite();
    write database;
    //check out-wake up waiting readers/writers;
    DoneWrite();
}
```

```
Private Database::StartWrite() {
    lock.Acquire();
    while ((AW+AR) > 0) {
        WW++;
        okToWrite.wait(&lock);
        WW--;
    }
    AW++;
    lock.Release();
}
```

```
Private Database::DoneWrite() {
    lock.Acquire();
    AW--;

    lock.Release();
}
```

写者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Write() {
    //Wait until no readers/writers;
    StartWrite();
    write database;
    //check out-wake up waiting readers/writers;
    DoneWrite();
}
```

```
Private Database::StartWrite() {
    lock.Acquire();
    while ((AW+AR) > 0) {
        WW++;
        okToWrite.wait(&lock);
        WW--;
    }
    AW++;
    lock.Release();
}
```

```
Private Database::DoneWrite() {
    lock.Acquire();
    AW--;
    if (WW > 0) {
        okToWrite.signal();
    }

    lock.Release();
}
```

写者实现

```
AR = 0;    // # of active readers
AW = 0;    // # of active writers
WR = 0;    // # of waiting readers
WW = 0;    // # of waiting writers
Lock lock;
Condition okToRead;
Condition okToWrite;
```

```
Public Database::Write() {
    //Wait until no readers/writers;
    StartWrite();
    write database;
    //check out-wake up waiting readers/writers;
    DoneWrite();
}
```

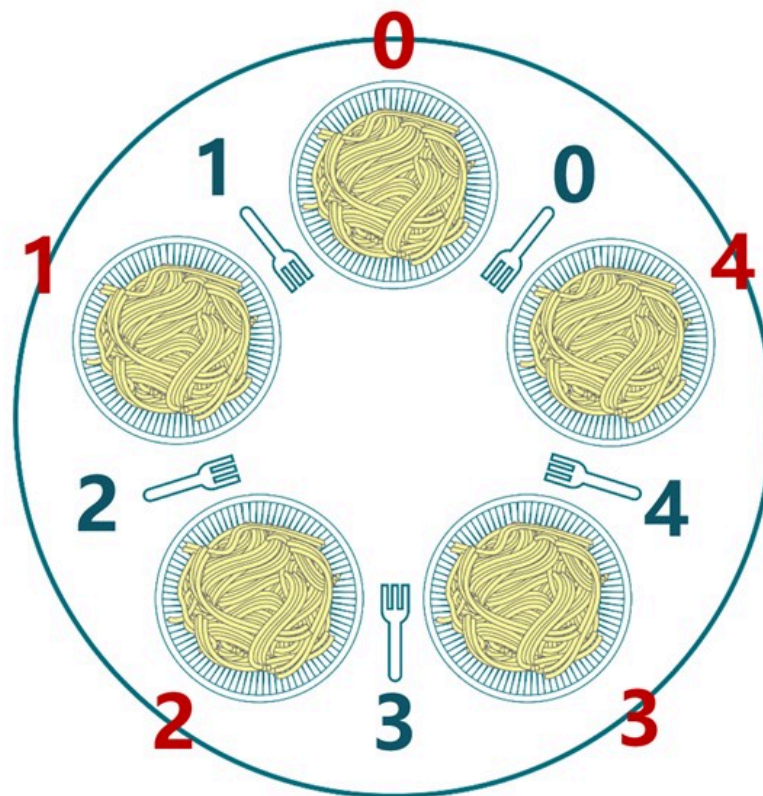
```
Private Database::StartWrite() {
    lock.Acquire();
    while ((AW+AR) > 0) {
        WW++;
        okToWrite.wait(&lock);
        WW--;
    }
    AW++;
    lock.Release();
}
```

```
Private Database::DoneWrite() {
    lock.Acquire();
    AW--;
    if (WW > 0) {
        okToWrite.signal();
    }
    else if (WR > 0) {
        okToRead.broadcast();
    }
    lock.Release();
}
```

哲学家就餐问题

- 1965年Dijkstra提出。
- 5个哲学家围绕一张圆桌而坐
- 桌子上放着5支叉子
- 每两个哲学家之间放一支叉子
- 哲学家的动作包括思考和进餐
- 进餐时需同时用左右两边的叉子
- 思考时将两支叉子放回原处

如何保证哲学家们的动作有序进行？
如：不出现有人永远拿不到叉子



哲学家就餐问题 方案1

```
#define N 5 // 哲学家个数
void philosopher(int i) // 哲学家编号: 0 - 4

while(TRUE)
{
    think(); // 哲学家在思考
    take_fork(i); // 去拿左边的叉子
    take_fork((i + 1) % N); // 去拿右边的叉子
    eat(); // 吃面条中....
    put_fork(i); // 放下左边的叉子
    put_fork((i + 1) % N); // 放下右边的叉子
}
```

哲学家就餐问题 方案2

```
#define N 5 // 哲学家个数

void philosopher(int i) // 哲学家编号: 0 - 4
while(1) // 去拿两把叉子
{
    take_fork(i); // 去拿左边的叉子
    if(fork((i+1)%N)) { // 右边叉子还在吗
        take_fork((i + 1) % N); // 去拿右边的叉子
        break; // 两把叉子均到手
    } else { // 右边叉子已不在
        put_fork(i); // 放下左边的叉子
        wait_some_time( ); // 等待一会儿
    }
}
```

对拿叉子的过程进行了改进，但仍然有问题

哲学家就餐问题 方案3

```
#define N 5 // 哲学家个数
void philosopher(int i) // 哲学家编号: 0 - 4
{
    while(1) // 去拿两把叉子
    {
        take_fork(i); // 去拿左边的叉子
        if(fork((i+1)%N)) { // 右边叉子还在吗
            take_fork((i + 1) % N); // 去拿右边的叉子
            break; // 两把叉子均到手
        } else { // 右边叉子已不在
            put_fork(i); // 放下左边的叉子
            wait_random_time( ); // 等待随机长时间
        }
    }
}
```

等待时间随机变化，可行，但非万全之策。

哲学家就餐问题 方案4

```
semaphore  mutex                // 互斥信号量, 初值1
void  philosopher(int  i)        // 哲学家编号i: 0-4

    while(TRUE) {
        think( );                // 哲学家在思考
        P(mutex);                // 进入临界区
        take_fork(i);            // 去拿左边的叉子
        take_fork((i + 1) % N);  // 去拿右边的叉子
        eat( );                  // 吃面条中...
        put_fork(i);             // 放下左边的叉子
        put_fork((i + 1) % N);   // 放下右边的叉子
        V(mutex);                // 退出临界区
    }
```

互斥保护。正确，但每次只允许一人就餐

方案4的缺点：

它把就餐（而不是叉子）看成是必须互斥访问的临界资源，因此会造成（叉子）资源的浪费。

从理论上说，如果有五把叉子，应允许两个不相邻的哲学家同时进餐。

哲学家就餐问题

思路(1) 哲学家自己怎么来解决这个问题？

指导原则：要么不拿，要么就拿两把叉子。

- S1 思考中...
- S2 进入饥饿状态；
- S3 如果左邻居或右邻居正在进餐，等待；否则转S4
- S4 拿起两把叉子；
- S5 吃面条...
- S6 放下左边的叉子；
- S7 放下右边的叉子；
- S8 新的循环又开始了，转S1

哲学家就餐问题

思路(2) 计算机程序怎么来解决这个问题？

指导原则：不能浪费CPU时间；进程间相互通信。

- S1 思考中...
- S2 进入饥饿状态；
- S3 如果左邻居或右邻居正在进餐，进程进入阻塞态；
否则转S4
- S4 拿起两把叉子；
- S5 吃面条...
- S6 放下左边的叉子，看看左邻居现在能否进餐
(饥饿状态、两把叉子都在)，若能则唤醒之；
- S7 放下右边的叉子，看看右邻居现在能否进餐，
(饥饿状态、两把叉子都在)，若能，唤醒之；
- S8 新的一天又开始了，转S1

哲学家就餐问题

思路(3) 怎么样来编写程序？

1. 必须有数据结构，来描述每个哲学家的当前状态；
2. 该状态是一个临界资源，各个哲学家对它的访问应该互斥地进行——进程互斥；
3. 一个哲学家吃饱后，可能要唤醒它的左邻右舍，两者之间存在着同步关系——进程同步；

哲学家就餐问题

1. 必须有一个数据结构，来描述每个哲学家的当前状态；

```
#define N      5           // 哲学家个数
#define LEFT   i           // 第i个哲学家的左邻居
#define RIGHT  (i+1)%N     // 第i个哲学家的右邻居
#define THINKING 0        // 思考状态
#define HUNGRY  1          // 饥饿状态
#define EATING  2          // 进餐状态
int state[N];              // 记录每个人的状态
```

2. 该状态是一个临界资源，对它的访问应该互斥地进行

```
semaphore mutex;           // 互斥信号量，初值1
```

3. 一个哲学家吃饱后，可能要唤醒邻居，存在着同步关系

```
semaphore s[N];            // 同步信号量，初值0
```

哲学家就餐问题

函数philosopher的定义

```
void philosopher(int i)    // i的取值: 0到N-1
{
    while(TRUE)            // 封闭式循环
    {
        S1 → think( );    // 思考中...
        S2 -S4 → take_forks(i); // 拿到两把叉子或被阻塞
        S5 → eat( );      // 吃面条中...
        S6 -S7 → put_forks(i); // 把两把叉子放回原处
    }
}
```

哲学家就餐问题

函数take_forks的定义

```
// 功能：要么拿到两把叉子，要么被阻塞起来。  
void take_forks(int i)      // i的取值：0到N-1  
{  
    state[i] = HUNGRY;      // 我饿了!  
}
```

哲学家就餐问题

函数take_forks的定义

// 功能：要么拿到两把叉子，要么被阻塞起来。

```
void take_forks(int i)           // i的取值：0到N-1
{
    state[i] = HUNGRY;           // 我饿了!
    test_take_left_right_forks(i); // 试图拿两把叉子
}
```

哲学家就餐问题

函数take_forks的定义

```
// 功能：要么拿到两把叉子，要么被阻塞起来。  
void take_forks(int i)           // i的取值：0到N-1  
{  
    state[i] = HUNGRY;           // 我饿了！  
    test_take_left_right_forks(i); // 试图拿两把叉子  
  
    P(s[i]);                     // 没有叉子便阻塞  
}
```

哲学家就餐问题

函数take_forks的定义

// 功能：要么拿到两把叉子，要么被阻塞起来。

```
void take_forks(int i)           // i的取值：0到N-1
{
    P(mutex);                    // 进入临界区
    state[i] = HUNGRY;           // 我饿了!
    test_take_left_right_forks(i); // 试图拿两把叉子
    V(mutex);                    // 退出临界区
    P(s[i]);                      // 没有叉子便阻塞
}
```

哲学家就餐问题

函数test_take_left_right_forks的定义

```
void test_take_left_right_forks(int i)    //i: 0到N-1
{
    if(state[i] == HUNGRY &&           // i: 我自己, or 其他人
        state[LEFT] != EATING &&
        state[RIGHT] != EATING)
    {
        state[i] = EATING;              // 两把叉子到手
        V(s[i]);                         //通知第i人可以吃饭了
    }
```

哲学家就餐问题

函数put_forks的定义

// 功能： 把两把叉子放回原处，并在需要的时候，
// 去唤醒左邻右舍。

```
void put_forks(int i)           // i的取值：0到N-1
{
    state[i] = THINKING;       // 交出两把叉子
```

哲学家就餐问题

函数put_forks的定义

// 功能： 把两把叉子放回原处，并在需要的时候，
// 去唤醒左邻右舍。

```
void put_forks(int i) // i的取值：0到N-1
{
    state[i] = THINKING; // 交出两把叉子
    test_take_left_right_forks(LEFT); // 看左邻居能否进餐
    test_take_left_right_forks(RIGHT); // 看右邻居能否进餐
}
```

哲学家就餐问题

函数test_take_left_right_forks的定义

```
void test_take_left_right_forks(int i) //i: 0到N-1
{
    if(state[i] == HUNGRY && // i: 我自己, or 其他人
        state[LEFT] != EATING &&
        state[RIGHT] != EATING)
    {
        state[i] = EATING; // 两把叉子到手
        V(s[i]); //通知第i人可以吃饭了
    }
}
```

哲学家就餐问题

函数put_forks的定义

```
// 功能： 把两把叉子放回原处，并在需要的时候，  
// 去唤醒左邻右舍。
```

```
void put_forks(int i) // i的取值：0到N-1  
{  
    P(mutex); // 进入临界区  
    state[i] = THINKING; // 交出两把叉子  
    test_take_left_right_forks(LEFT); // 看左邻居能否进餐  
    test_take_left_right_forks(RIGHT); // 看右邻居能否进餐  
    V(mutex); // 退出临界区  
}
```

哲学家就餐问题

函数philosopher的定义

```
void philosopher(int i)      // i的取值: 0到N-1
{
    while(TRUE)              // 封闭式循环
    {
        S1  → think( );      // 思考中...
        S2 -S4 → take_forks(i); // 拿到两把叉子或被阻塞
        S5  → eat( );         // 吃面条中...
        S6 -S7 → put_forks(i); // 把两把叉子放回原处
    }
}
```

哲学家就餐问题

函数philosopher的定义

```
void philosopher(int i)    // i的取值: 0到N-1
{
    while(TRUE)            // 封闭式循环
    {
        S1  —————> think( );                // 思考中...
        S2 -S4 —————> take_forks(i);          // 拿到两把叉子或被阻塞
        S5  —————> eat( );                    // 吃面条中...
        S6 -S7 —————> put_forks(i);           // 把两把叉子放回原处
    }
}
```

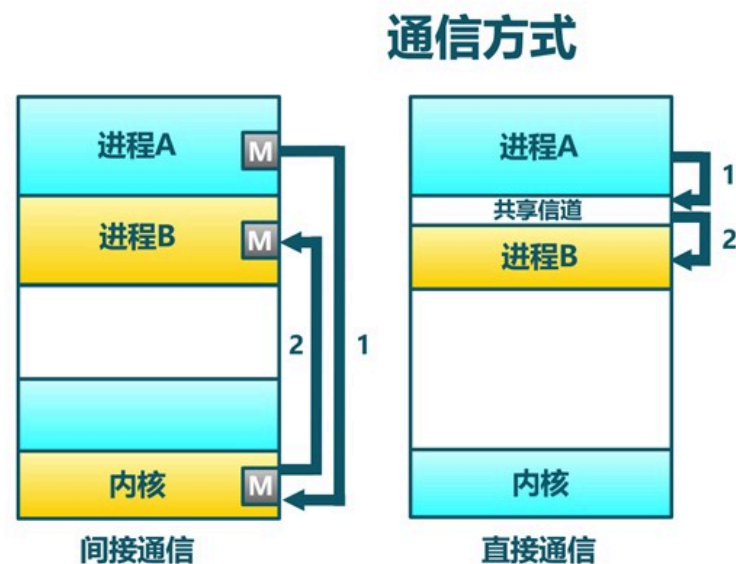
think(), eat() 是否需要进一步实现? 如需要, 如何实现?

3.7 进程通信

进程间通信的需求

- 挑战：单个程序的功能有限
- IPC的目标：多进程协作完成复杂应用需求
 - 功能模块化
 - 程序之间相对隔离
 - 多个程序合作可完成复杂任务

进程间通信的定义：进程间通过数据交换（共享或传递）进行**交互**的行为



进程间的交互关系

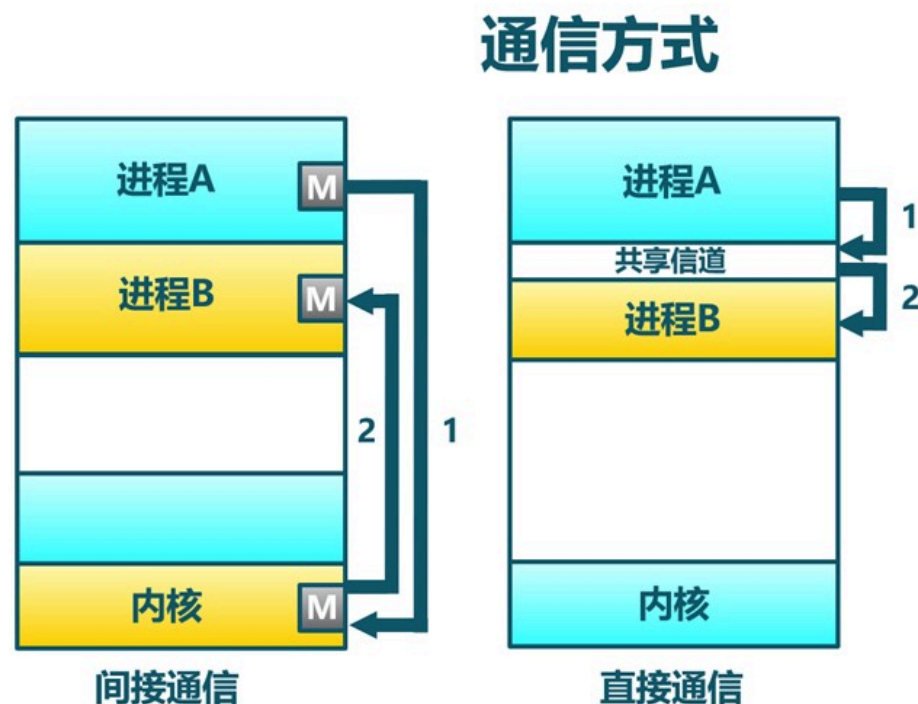
- 独立进程：与其它进程无交互
- 协作进程：两个或多个进程之间有交互
 - 发送者 接收者 / 客户端 服务端

```
> cat README.md | grep rcore
$ git clone https://github.com/rcore-os/rCore-Tutorial-v3.git
...
* [x] expand the fs image size generated by `rcore-fs-fuse` to 128MiB
```

- `grep` 依赖 `cat`
 - `grep` 等 `cat` 产生的输出作为其输入，来匹配字符串

进程通信方式

- **直接通信**：两个进程间不需要通过内核的中转，就可以相互传递信息
- **间接通信**：两个进程间通过系统调用和内核的中转，来相互传递消息



IPC机制

进程间能共享或传递数据就算是进程间通信。

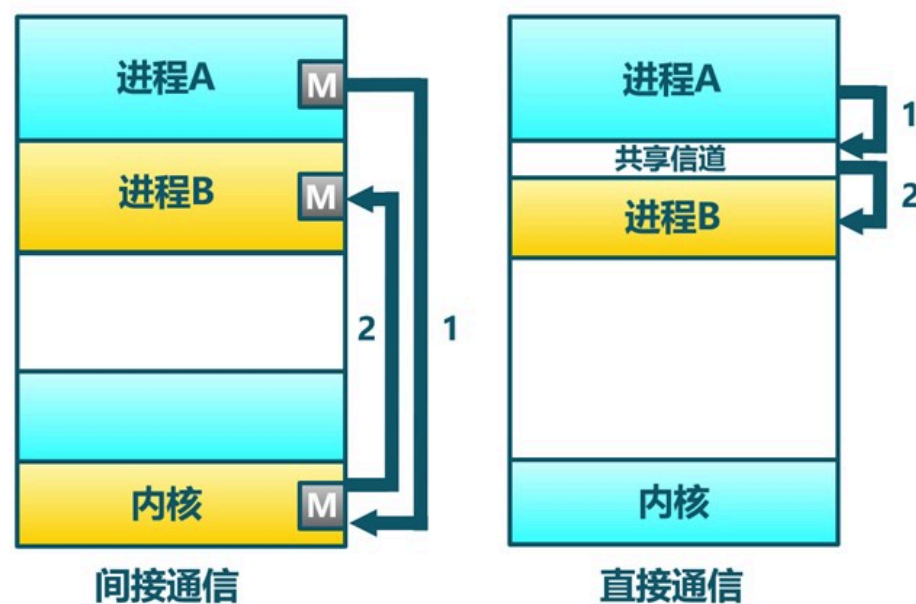
IPC机制	含义	通信方式
信号 (Signal)	异步发送信号给进程处理	间接通信
管道 (Pipe)	单方向传输字节流	间接通信
消息队列 (Message Queue)	通过队列中转收/发消息	间接通信
套接字 (Socket)	多/单机进程间网络通信	间接通信
共享内存 (Shared Memory)	多个进程共享一块物理内存	直接通信
文件 (File)	多个进程可访问同一文件	间接通信

消息传递的基本接口

- 发送(send)消息
- 接收(recv)消息
- 远程过程调用(RPC)
- 回复(reply)消息

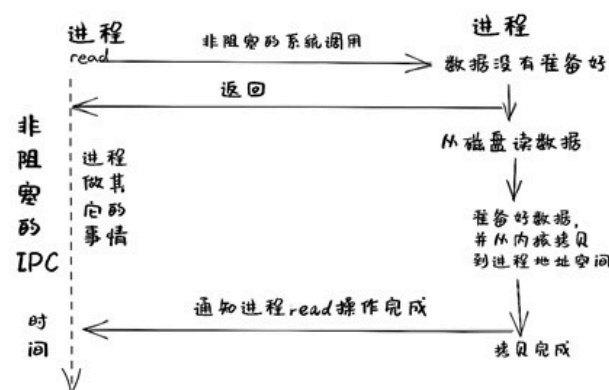
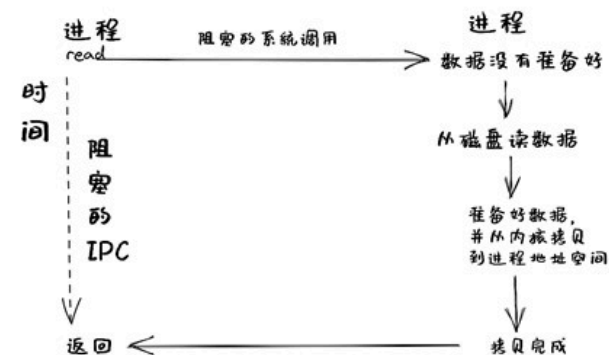
Remote Procedure Call,
RPC = send + recv

通信方式



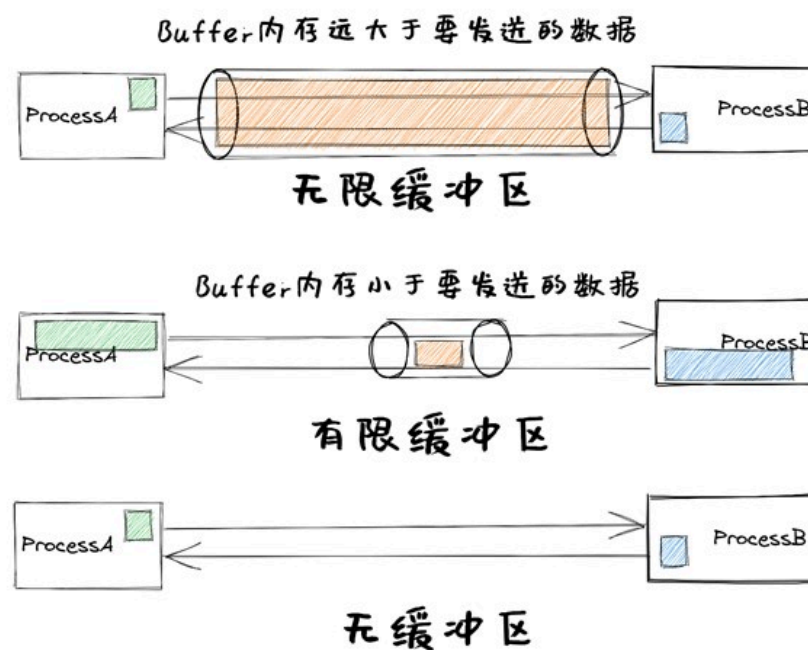
阻塞或非阻塞通信

- 阻塞通信:
 - 阻塞发送、阻塞接收
- 非阻塞通信
 - 非阻塞发送、非阻塞接收



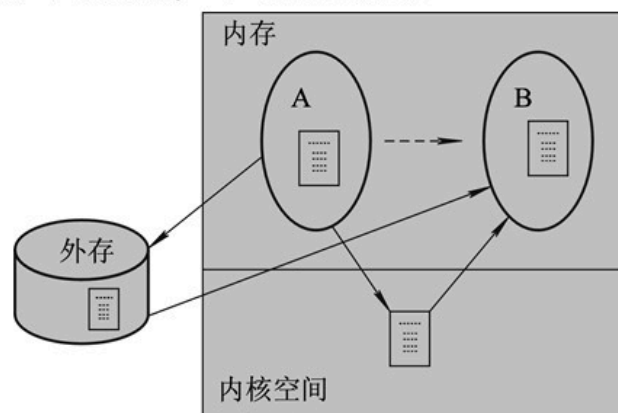
IPC的缓冲方式

- **无限容量**: 发送方不需要等待
- **有限容量**: 通信链路缓冲队列满时, 发送方必须等待
- **0容量**: 发送方必须等待接收方



3.7 进程通信

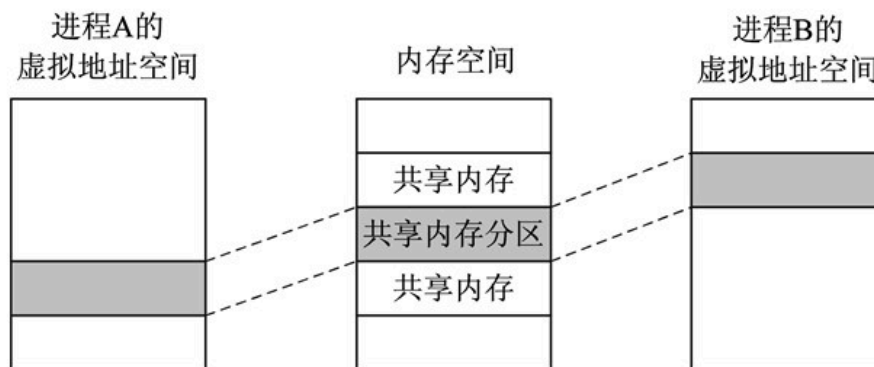
- 进程通信方案：
- (1) 发送、接收双方经过事先约定，利用磁盘等外存设备由发送进程A把要交换的数据写入外存的指定区域，接收进程B从该区域读取数据。这种方案实现简单，只需要操作系统的文件系统即可，但通信过程却需要I/O操作。
- (2) 利用内核运行在核心态的特点，发送进程A通过内核程序将数据写入内核空间指定的区域，接收进程B则通过另一组内核程序从内核指定区域中读取数据，并写入接收进程B的内存数据空间，从而实现数据从进程A传送到进程B。这种方案需要操作系统专门的系统调用来实现，数据传送不需要I/O操作（全部在内存中完成），速度快。



- 进程通信分为如下两种：
- (1) **低级通信**。进程之间一次只能传送很少的信息。低级通信的优点是速度快；缺点是传递的信息量少、效率低，通信过程对用户不透明，因此编程较复杂。低级通信主要是作为互斥与同步的工具来使用，所以我们也称P、V原语称为低级通信原语。
- (2) **高级通信**。进程间一次可以传送大量的信息。优点是通信效率高，通信过程对用户透明，编程相对较简单。高级通信的目的不仅是为了控制进程的执行速度，而且是为了交换大量的信息。
- 进程间的**高级通信**分为如下3种：
- ① 共享内存通信方式。通信双方利用共享的内存区来实现进程间通信。
- ② 共享文件通信方式。通信双方利用共享一个文件来实现进程间通信。该通信方式又称为管道通信，而被共享的文件称为管道。
- ③ 消息传递通信方式。利用操作系统提供的消息传递系统来实现进程间通信，进程间以消息为单位来进行信息交换。

• 1.共享内存通信方式

- 含义：指在内存中划出一块内存区作为共享数据区，称为共享内存分区，要通信的进程双方将自己的虚拟地址空间映射到共享内存分区上（如下图）。
- 通信时，发送进程将需要交换的信息写入该共享内存分区中，接收进程从该共享内存分区中读取信息，从而实现进程之间的通信。
- 是进程之间最快捷、最有效的一种通信方式。UNIX、Windows、OS/2等操作系统都采用了这种通信方式。

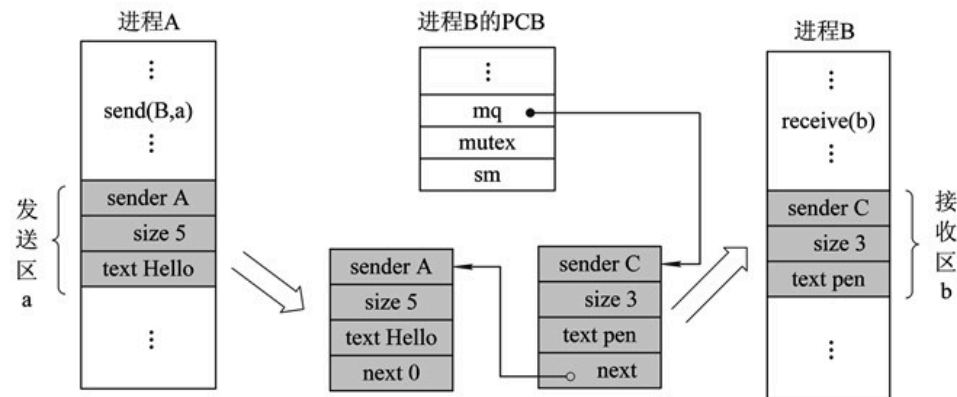


• 2.消息缓冲通信方式

- 消息缓冲通信属于直接通信方式。
- 若采用消息缓冲通信方式，则发送进程利用发送命令（原语）直接将信息发送到接收进程的消息队列，而接收进程则利用接收命令（原语）从自己的消息队列中取出消息。进程间信息交换以消息为单位。
- 对于发送者，在通信中存在两种可能的行为：① 发送完消息后不等消息被接收者接收就继续前进；② 发送完消息后阻塞自己直到收到接收者的回答消息后才继续前进。
- 对于接收者，也存在两种可能的行为：① 若有消息则接收这个消息后继续前进，若无消息则必须等待消息的到来（即阻塞自己）；② 若有消息则接收这个消息后继续前进，若无消息则放弃接收消息而继续前进。
- 进程之间的通信分为单向通信和双向通信。

- **消息缓冲机制**通信过程描述如下：
 - (1) 发送者在发送消息前，先在自己的内存空间设置一个发送区，把欲发送的消息填入其中。
 - (2) 发送者申请一个消息缓冲区，将已准备好的消息从发送区送到该消息缓冲区，并将发送者进程的名字、消息的开始地址以及消息的长度（通常以字节或字为单位）等信息填入该消息缓冲区中，然后把该消息缓冲区挂到接收进程的消息链上。
 - (3) 接收者在接收消息前先在自己的内存空间设置相应的接收区。
 - (4) 接收者摘下消息链上的第一条消息，将该消息从消息缓冲区复制到接收区，然后释放该消息缓冲区。

- 消息缓冲区数据结构描述如下:
- `struct messagebuffer`
- {
- `int sender;` //发送进程标识符
- `int size;` //消息长度
- `char text[MAXSIZE];` //消息正文, MAXSIZE为数组text的长度
- `struct messagebuffer *next;` // next为指向下一个消息缓冲区的指针
- }



3.7 进程通信

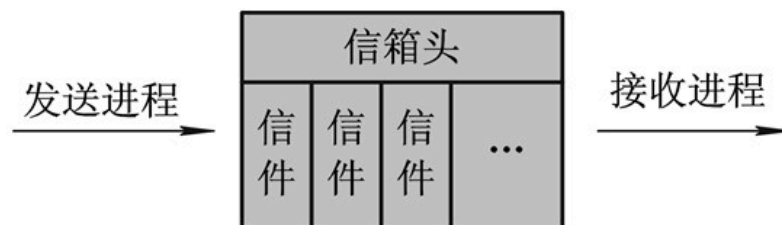
- 发送原语send可描述如下:
- ```
void send(receiver, a)
{
 lock out interrupts; //关中断
 getbuffer(a.size, i); //根据消息a的长度size申请一个空消息缓冲区i
 i.sender=a.sender; //将发送区a的内容复制到消息缓冲区i
 i.size=a.size;
 i.text=a.text;
 i.next=NULL;
 getid(PCBset, receiver, j); //从PCB集合中获得接收进程的标识符j
 P(j.mutex); //互斥使用j的消息队列
 Insert(j.mq, i); //将消息缓冲区i挂接在进程j的消息队列的队尾
 V(j.mutex); //释放对j的消息队列的使用权, 允许其他进程访问
 V(j.sm); //进程j的消息个数加1, 若进程j因等待该消息被阻塞则唤醒它
 unlock interrupts; //开中断
}
```

- 接收原语receive可描述如下:
- `void receive(b)`
- `{`
- `lock out interrupts;`     //关中断
- `j=internal name;`       //将接收进程的內部标识符存放到变量j中
- `P(j.sm);`               //若j的消息队列中有消息缓冲区则继续, 否则阻塞进程j
- `P(j.mutex);`           //互斥使用j的消息队列
- `Remove(j.mq, i);`       //从j的消息队列中取出第一个消息缓冲区i
- `V(j.mutex);`           //释放对j的消息队列的使用权, 允许其他进程访问
- `b.sender=i.sender;`   //将消息缓冲区i的内容复制到消息接收区b中
- `b.size=i.size;`
- `b.text=i.text;`
- `Releasebuff(i);`       //释放消息缓冲区i占用的内存
- `unlock interrupts;`    //开中断
- `}`

## 3.7 进程通信

### • 3.信箱通信方式

- 含义：指进程之间的通信要借助于称为信箱的共享数据结构实体，来暂时存放发送进程发送给接收进程的消息。
- 采用信箱通信的最大好处是，发送方和接收方不必直接建立联系，也没有处理时间上的限制；发送方可以在任何时间发送信件，接收方也可以在任何时间取走信件。
- 目前人们常用的E-mail就是采用这种方式收发信件的。



- 信箱通信应具有以下同步规则：
- （1）若发送信件时信箱已满，则发送进程应转变成等待信箱状态，直到信箱有空信格时才被唤醒。
- （2）若取信件时信箱中已无信件，则接收进程转变成等待信件状态，直到有信件时才被唤醒。
- 以私用信箱为例描述信箱机制的通信过程：
- （1）接收者创建属于自己的私用信箱；
- （2）发送者产生一封信件（即一个消息）。
- （3）发送者把信件投入接收者的私用信箱。
- （4）接收者从自己的私用信箱中读取信件。

- 投递原语desposit描述如下:
- `void deposit(boxname, msg)`
- `{`
- `lock out interrupts;`         //关中断
- `P(emptynum);`             //有空格子否, 无则阻塞自己
- 选择标志位为空的格子;
- 把信件msg放入该空格子中;
- 置该格子的标志位为满;
- `V(mesnum);`             //信件多了一个, 若有接收者进程被阻塞则唤醒之
- `unlock interrupts;`         //开中断
- `}`

- 读取原语remove描述如下:
- `void remove(boxname, msg)`
- {
- `lock out interrupts;`         //关中断
- `P(mesnum);`                 //有信件否, 无则阻塞自己
- 选择标志位为满的格子;
- 把该满格子中的信件放入msg中;
- 置该格子的标志位为空;
- `V(emptynum);`             //空格子多了一个, 若有发送者进程被阻塞则唤醒之
- `unlock interrupts;`         //开中断
- }

## 3.7 进程通信

### • 4.管道通信方式

- 由于内存容量有限，使用共享内存通信方式交换的信息量受到一定限制，为了传输大量数据，可以采用管道通信（即共享文件通信）的方式。
- 管道，是指连接在两个进程之间的一个打开的共享文件，也称为pipe文件，专门用于进程之间进行数据通信。发送进程可以源源不断地从管道一端写入数据流，且每次写入的长度是可变的；接收进程可以从管道的另一端读出数据，读出的长度也是可变的。



管道的特殊处：

- (1) 管道专门用于通信。
- (2) 管道只能单向传送数据。
- (3) 在对管道进行读写操作过程中，发送者进程和接收者进程所需要的同步与互斥都由系统自动进行，即对用户是透明的。

- **管道分类：**
  1. 无名管道
    - 无名管道是利用系统调用pipe建立起来的无路径名的无名文件，并且是一个临时文件。无名管道在物理上则由文件系统的高速缓冲区（高速缓存Cache中的一个区域）构成，而且很少启动外部设备。
  2. 有名管道
    - 与无名管道是一个临时文件不同，有名管道被建立后在磁盘上有一个对应的目录项和索引节点。即有名管道是具有路径名并可在文件系统中长期存在的真实文件，它不能与文件系统中的任何文件重名，并且在文件系统中也可以看到该有名管道。此外，对有名管道的访问也与一般文件一样，需要先用系统调用open打开。
- **管道通信的优点：** 传送数据量大，并且管道通信机制中的互斥与同步都由操作系统自动进行（对用户透明），使用方便；
- **缺点：** 通信速度较慢。

**填空题 1分**

**(考研真题) 假设系统中有下述 3 种解决死锁的方法：**

- (1) 银行家算法；**
- (2) 检测死锁，终止处于死锁状态的进程，释放该进程所占有的资源；**
- (3) 资源预分配。**

**简述上述哪种方法允许最大的并发性？请按“并发性”从大到小对上述 3 种方法进行排序。**

**[填空1]**

①题中给出的3种方法中，检测死锁能允许更多的进程无须等待地向前推进，并发性最大。因为该方法允许进程最大限度地申请并分配资源，直至出现死锁，再由系统解决死锁。

②银行家算法允许进程自由申请资源，只是在某个进程申请资源时会检查系统是否处于安全状态，若是，则可立即分配；若不是，则拒绝。其并发性的的大小次于检测死锁这一方法。

③最后是资源预分配，因为此方法要求进程在运行之前申请所需的全部资源，这会使许多进程因申请不到全部资源而无法开始，得到部分资源的进程也会因未得到全部资源而不释放已占用的资源，所以此方法会导致资源浪费。

因此，上述3种方法按并发性从大到小排序为：检测死锁、银行家算法、资源预分配。

单选题 1分

若有4个进程共享同一程序段，而且每次最多允许3个进程进入该程序段，则信号量的变化范围是（ ）。

- ☐ A 3, 2, 1, 0
- ☒ B 3, 2, 1, 0, -1
- ☐ C 4, 3, 2, 1, 0
- ☐ D 2, 1, 0, -1, -2

**填空题** 1分

**(考研真题)** 有 $m$  个进程共享同一临界资源，若使用信号量机制实现对某个临界资源的互斥访问，请求出信号量的变化范围 **[填空1]**。

## 主观题 10分

(考研真题) 由5个进程组成进程集合 $P=\{P_0, P_1, P_2, P_3, P_4\}$ , 系统中有3类资源A, B, C, 假设在某时刻有表1-3-4所示的进程资源分配情况。

表1-3-4 进程资源分配情况

| 进程    | Allocation | Max   | Available   |
|-------|------------|-------|-------------|
|       | A B C      | A B C | A B C       |
| $P_0$ | 0 0 3      | 0 0 4 | $x \ y \ z$ |
| $P_1$ | 1 0 0      | 1 7 5 |             |
| $P_2$ | 1 3 5      | 2 3 5 |             |
| $P_3$ | 0 0 2      | 0 6 4 |             |
| $P_4$ | 0 0 1      | 0 6 5 |             |

请问当  $x, y, z$  取下列值时, 系统是否处于安全状态?

(1) 1, 4, 0; (2) 0, 6, 2; (3) 1, 1, 1; (4) 0, 4, 7。

【参考答案】由银行家算法中Need、Max、Allocation间的关系可得：

$$\text{Need} = \text{Max} - \text{Allocation} = \begin{bmatrix} 0 & 0 & 4 \\ 1 & 7 & 5 \\ 2 & 3 & 5 \\ 0 & 6 & 4 \\ 0 & 6 & 5 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 3 \\ 1 & 0 & 0 \\ 1 & 3 & 5 \\ 0 & 0 & 2 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 7 & 5 \\ 1 & 0 & 0 \\ 0 & 6 & 2 \\ 0 & 6 & 4 \end{bmatrix}。$$

(1) 当Available为(1, 4, 0)时，根据Need矩阵可知，可满足P<sub>2</sub>的需求；P<sub>2</sub>结束后释放资源，Available为(2, 7, 5)可满足P<sub>0</sub>、P<sub>1</sub>、P<sub>3</sub>、P<sub>4</sub>中任一进程的需求，因此系统不会出现死锁，处于安全状态。

(2) 当Available为(0, 6, 2)时，可满足进程P<sub>0</sub>、P<sub>3</sub>的需求；进程结束后释放资源，Available为(0, 6, 7)，可满足P<sub>4</sub>的需求；P<sub>4</sub>结束后释放资源，Available为(0, 6, 8)，此时不能满足其余任一进程的需求，系统出现死锁，因此当前处在非安全状态。

(3) 当Available为(1, 1, 1)时，可满足进程P<sub>0</sub>、P<sub>2</sub>的需求；这两个进程结束后释放资源，Available为(2, 4, 9)，此时不能满足其余任一进程的需求，系统出现死锁，处于非安全状态。

(4) 当Available为(0, 4, 7)时，可满足P<sub>0</sub>的需求，进程结束后释放资源，Available为(0, 4, 10)，此时不能满足其余任一进程的需求，系统出现死锁，处于非安全状态。

# 第三章 (完)